

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN XÁC SUẤT VÀ THỐNG KÊ

PHÂN TÍCH CÁC THÔNG SỐ KỸ THUẬT VÀ
DỰ ĐOÁN GIÁ GPU

NHÓM MT02 – LỚP DL03 – HỌC KỲ 233

Giảng viên hướng dẫn: Th.S Nguyễn Kiều Dung

Họ và tên Sinh viên	Mã số Sinh viên	Nhiệm vụ	Phần trăm đóng góp
Nguyễn Trọng Nhân	2212370	Hồi quy đa bội	100%
Nguyễn Hoàng Gia Bảo	2310242	Kiểm định hai mẫu	100%
Trương Hoàng Danh	2310423	Kiểm định một mẫu	100%
Bùi Đăng Khoa	2211581	Thực hiện tiền xử lí số liệu	100%
Nguyễn Văn Thịnh	2213301	Anova hai nhân tố	100%
Võ Trần Đăng Bách	2210188	Anova một nhân tố	100%
Trần Minh Trí	2313626	Thực hiện thống kê mô tả	100%



Mục lục

1	Tổng quan dữ liệu	3
2	Cơ sở lý thuyết	4
2.1	Kiểm định giả thuyết trung bình	4
2.1.1	Ước lượng khoảng tin cậy 1 mẫu	4
2.1.2	Kiểm định trung bình hai mẫu	4
2.2	Phân tích phương sai (phân tích ANOVA)	7
2.2.1	Phân tích phương sai 1 yếu tố (one-way ANOVA)	7
2.2.2	Phân tích phương sai 2 yếu tố (Two-way ANOVA)	8
2.3	Hồi quy tuyến tính đa biến	10
3	Tiền xử lý số liệu.	11
3.1	Đọc dữ liệu.	11
3.2	Trích xuất dữ liệu.	12
3.3	Xử lý định dạng dữ liệu.	13
3.4	Xử lý dữ liệu khuyết thiếu.	18
4	Thống kê mô tả	20
4.1	Các đặc trưng của mẫu	20
4.2	Biểu đồ và đồ thị	20
4.2.1	Biểu đồ tần số	20
4.2.2	Phân phối chuẩn	23
4.2.3	Mối liên hệ giữa các biến	26
5	Thống kê suy diễn	28
5.1	Tìm khoảng tin cậy một mẫu	28
5.1.1	Mục tiêu	28
5.1.2	Bài toán	28
5.1.3	Thực hiện	28
5.1.4	Kết quả	28
5.1.5	Nhận xét	29
5.1.6	Kết luận	29
5.2	Kiểm định hai mẫu	30
5.2.1	Mục tiêu	30
5.2.2	Bài toán	30
5.2.3	Nhận xét bài toán	30
5.2.4	Kiến thức R	30
5.2.5	Tiến hành	31
5.2.6	Kết quả	31
5.2.7	Nhận xét	32
5.2.8	Kết luận	32
5.3	Phân tích phương sai ANOVA 1 yếu tố	33
5.3.1	Mục tiêu	33
5.3.2	Bài toán	33
5.3.3	Kiểm tra giả thiết	33
5.3.4	Quy trình giải quyết bài toán	33
5.3.5	Kết luận	35
5.4	Phân tích phương sai 2 yếu tố (two-way ANOVA)	36
5.4.1	Bài toán	36
5.4.2	Kiểm tra điều kiện cho dữ liệu	36
5.4.3	Tiến hành phân tích	36



5.4.4	Kết luận	40
5.5	Hồi quy tuyến tính đa biến	40
5.5.1	Tiền xử lý số liệu	41
5.5.2	Xây dựng mô hình hồi quy	41
5.5.3	Ước lượng khoảng tin cậy của các hệ số	43
5.5.4	Kiểm tra các giả định của mô hình	44
5.5.5	Dự đoán giá	47
6	Tổng kết	47
7	Thảo luận và mở rộng	47



1 Tổng quan dữ liệu

Bộ xử lý đồ họa (GPU) là một mạch điện tử có khả năng thực hiện các phép toán ở tốc độ cao. Các tác vụ điện toán như kết xuất đồ họa, máy học (ML) và chỉnh sửa video yêu cầu áp dụng các phép toán tương tự trên một tập dữ liệu lớn. Thiết kế của GPU cho phép thực hiện cùng một phép toán trên nhiều giá trị dữ liệu song song. Điều này làm tăng hiệu quả xử lý của GPU cho nhiều tác vụ điện toán chuyên sâu.

Tập tin All_GPUs.csv chứa 34 thông số của hơn 3400 bộ xử lý đồ họa GPU, với một số thống số như:

- Name: Tên mẫu GPU.
- Best Resolution: Thuộc tính cho biết độ phân giải của máy tính để GPU có thể hoạt động hiệu quả nhất. Độ phân giải (resolution) là số lượng điểm ảnh (pixels) trên màn hình mà GPU có khả năng hiển thị. Độ phân giải được biểu thị bằng hai giá trị: chiều rộng (số pixel theo chiều ngang) và chiều cao (số pixel theo chiều dọc). Ví dụ, độ phân giải thông dụng là 1920x1080, tức là có 1920 pixel theo chiều ngang và 1080 pixel theo chiều dọc.
- Manufacturer: Công ty hoặc tổ chức sản xuất và phân phối GPU. Các nhà sản xuất GPU nổi tiếng như Nvidia, AMD, Intel,... Mỗi nhà sản xuất có sản phẩm GPU riêng của họ với các đặc điểm và tính năng riêng biệt.
- Core Speed [MHz]: Là tốc độ hoạt động của các lõi xử lý (cores) trên GPU. Đây là một trong những thông số quan trọng để đánh giá hiệu năng của GPU. Thông số này được sử dụng để đo lường khả năng xử lý tính toán của GPU. Tốc độ lõi cao hơn thường cho phép GPU thực hiện nhiều phép tính trong một khoảng thời gian ngắn hơn, cải thiện hiệu năng tổng thể.
- Max Power [Watts]: Thể hiện lượng điện năng tối đa mà GPU sử dụng. Giới hạn công suất này là một tính năng cho phép người dùng kiểm soát lượng điện năng tiêu thụ vào card đồ họa.
- Memory [MB hoặc GB]: Dung lượng bộ nhớ truy cập đồ họa của GPU. Đây là một trong những thuộc tính ảnh hưởng trực tiếp đến khả năng xử lý đồ họa của máy tính. Dung lượng này cho biết khả năng lưu trữ các dữ liệu và tài nguyên đồ họa mà GPU có thể sử dụng trong quá trình hoạt.
- Memory Speed [MHz]: Đo lường tốc độ đọc/ghi của GPU vào bộ nhớ máy tính. Giá trị này càng cao, chứng tỏ GPU có khả năng truy cập vào vùng nhớ để truy xuất hoặc lưu trữ dữ liệu.
- Memory Bandwidth [GB/giây]: Đo lường khả năng truy xuất dữ liệu từ bộ nhớ của GPU. Bảng thông bộ nhớ càng cao, GPU có khả năng nhanh chóng truy cập và đọc/ghi dữ liệu từ bộ nhớ, tăng hiệu suất trong các tác vụ.
- Release Date: Ngày phát hành mẫu GPU.
- Release Price: Giá phát hành mẫu GPU.

Ngoài ra, tập tin còn chứa các thông số khác như Architecture (Kiểu kiến trúc xây dựng GPU), Pixel Rate (Tỉ lệ điểm ảnh), PSU (Mức điện năng sử dụng),...



2 Cơ sở lý thuyết

2.1 Kiểm định giả thuyết trung bình

2.1.1 Ước lượng khoảng tin cậy 1 mẫu

Giả thuyết: Giả sử rằng một tổng thể có trung bình μ_0 , biết hoặc không biết phương sai σ^2 . Một mẫu có kích thước n , trung bình mẫu, và phương sai s^2 được chọn ra từ tổng thể. Dựa trên mẫu đã chọn, hãy tìm khoảng ước lượng giá trị của của tổng thể đó.

Các bước giải:

- Xác định thông số và loại phân phối: Xác định loại phân phối (như phân phối chuẩn, phân phối t, v.v.) và thông số của mẫu (như trung bình mẫu, độ lệch chuẩn mẫu).
- Tính toán ngưỡng sai số ϵ . Cụ thể:
 - Nếu X phân phối chuẩn, biết σ^2 : $\epsilon = z_{\alpha/2} * \sigma / \sqrt{n}$
 - Nếu chưa biết σ^2 : $\epsilon = t_{\alpha/2; (n-1)} * s / \sqrt{n}$Nếu X phân phối bất kỳ và mẫu lớn ($n \geq 30$):
 - * Áp dụng tương tự như trường hợp X phân phối chuẩn với biết σ^2 .
- Diễn giải kết quả và kết luận: Khoảng ước lượng sẽ cung cấp một khoảng giá trị mà trong đó có khả năng cao chứa giá trị thực của tham số tổng thể.

2.1.2 Kiểm định trung bình hai mẫu

- Hai mẫu được gọi là độc lập khi sự thay đổi của một mẫu không ảnh hưởng đến sự thay đổi của mẫu kia. Nói cách khác, sự kiện xảy ra ở mẫu này không làm ảnh hưởng đến mẫu kia, và ngược lại.
- Hai mẫu được gọi là tương ứng phụ thuộc theo cặp khi kích thước của chúng bằng nhau và có một mối liên quan hay sự phụ thuộc giữa chúng. Hay nói cách khác, sự kiện xảy ra ở mẫu này có thể ảnh hưởng đến sự kiện của mẫu kia, và ngược lại.

Để kiểm tra tính độc lập (hay phụ thuộc) giữa hai mẫu, ta sử dụng kiểm định Chi bình phương (Chi-square - χ^2).

Trong giới hạn bài tập lớn này với bài toán kiểm định hai mẫu, nhóm chỉ xét trường hợp hai mẫu độc lập nhau.

- Đặt giả thiết bài toán:
 - Tổng thể I có trung bình μ_1 , tổng thể II có trung bình μ_2 .
 - Biết (hoặc chưa biết) phương sai hai tổng thể σ_1^2, σ_2^2 .
 - Từ hai tổng thể, có hai mẫu kích thước lần lượt n_1 và n_2 độc lập, có trung bình lần lượt là $\bar{X}_1, \bar{X}_2$.Hãy kiểm định, với mức ý nghĩa α , giả thuyết trung bình hai tổng thể bằng nhau có đáng tin cậy không.
- Đặt giả thuyết kiểm định:
 - Gọi $\mu_D = \mu_1 - \mu_2$ là chênh lệch giữa trung bình hai tổng thể I và II.
Giả thuyết không H_0 : Trung bình hai tổng thể không có sự khác biệt.



$$\mu_D = 0$$

Giả thuyết đối H_1 : Tùy theo yếu tố cần kiểm định mà có thể đặt H_1 là

- $\mu_D \neq 0$: Trung bình hai tổng thể có sự khác biệt.
- $\mu_D < 0$: Trung bình tổng thể I nhỏ hơn trung bình tổng thể II.
- $\mu_D > 0$: Trung bình tổng thể I lớn hơn trung bình tổng thể II.

- Tìm miền bác bỏ RR:

Miền bác bỏ H_0 cho bài toán kiểm định trung bình hai mẫu độc lập phụ thuộc vào giả thuyết đối H_1 và dữ kiện đã biết. Cụ thể:

Dạng			H_1	Miền bác bỏ H_0
X_1, X_2 phân phối chuẩn	Biết σ_1^2, σ_2^2		$\mu_1 \neq \mu_2$	$(-\infty; -t_{\alpha/2}(n-1)) \cup (t_{\alpha/2}(n-1); +\infty)$
			$\mu_1 < \mu_2$	$(-\infty; -z_\alpha)$
			$\mu_1 > \mu_2$	$(z_\alpha; +\infty)$
	Chưa biết σ_1^2, σ_2^2	$\frac{s_1}{s_2} \in [0.5; 2]$	$\mu_1 \neq \mu_2$	$(-\infty; -t_{\alpha/2}(n_1 + n_2 - 2)) \cup (t_{\alpha/2}(n_1 + n_2 - 2); +\infty)$
			$\mu_1 < \mu_2$	$(-\infty; -t_\alpha(n_1 + n_2 - 2))$
			$\mu_1 > \mu_2$	$(t_\alpha(n_1 + n_2 - 2); +\infty)$
		$\frac{s_1}{s_2} \notin [0.5; 2]$	$\mu_1 \neq \mu_2$	$(-\infty; -t_{\alpha/2}(v)) \cup (t_{\alpha/2}(v); +\infty)$
			$\mu_1 < \mu_2$	$(-\infty; -t_\alpha(v))$
			$\mu_1 > \mu_2$	$(t_\alpha(v); +\infty)$
X_1, X_2 phân phối tùy ý Kích thước hai mẫu lớn ($n_1, n_2 \geq 30$)			$\mu_1 \neq \mu_2$	$(-\infty; -t_{\alpha/2}(n-1)) \cup (t_{\alpha/2}(n-1); +\infty)$
			$\mu_1 < \mu_2$	$(-\infty; -z_\alpha)$
			$\mu_1 > \mu_2$	$(z_\alpha; +\infty)$

Trong đó, bậc tự do v được tính:

$$v = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{\left(\frac{s_1^2}{n_1}\right)^2}{n_1 - 1} + \frac{\left(\frac{s_2^2}{n_2}\right)^2}{n_2 - 1}}$$

- Tính giá trị kiểm định thống kê: Hàm kiểm định thống kê cho bài toán kiểm định trung bình hai mẫu rơi vào nhiều trường hợp, tùy theo dữ kiện đã biết, cụ thể:



Dạng		Tiêu chuẩn kiểm định
X_1, X_2 phân phối chuẩn	Biết σ_1^2, σ_2^2	$Z_{qs} = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}}$
	Chưa biết σ_1^2, σ_2^2	$s_p^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}$ $T_{qs} = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_p^2}{n_1} + \frac{s_p^2}{n_2}}}$
		$T_{qs} = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$
X_1, X_2 phân phối tùy ý Kích thước hai mẫu lớn ($n_1, n_2 \geq 30$)	Biết σ_1^2, σ_2^2	$Z_{qs} = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}}$
	Chưa biết σ_1^2, σ_2^2	$Z_{qs} = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$

• Kết luận

- Nếu Z_{qs} (hoặc T_{qs}) $\in$ RR: bác bỏ giả thuyết H_0 , thừa nhận giả thuyết H_1 . Tức kết luận với mức ý nghĩa α , giả thuyết trung bình hai tổng thể bằng nhau là không đáng tin cậy.
- Nếu Z_{qs} (hoặc T_{qs}) $\notin$ RR: chưa đủ dữ liệu khẳng định H_0 sai. Tức với mức ý nghĩa α , chưa thể nói giả thuyết trung bình hai tổng thể bằng nhau là không đáng tin cậy.



2.2 Phân tích phương sai (phân tích ANOVA)

Phân tích phương sai (Analysis of Variance- ANOVA) là một mô hình dùng để xem xét sự biến động của một biến ngẫu nhiên định lượng chịu tác động trực tiếp của một hay nhiều yếu tố nguyên nhân (định tính). Mục đích chính của ANOVA là để xác định xem những biến động trong dữ liệu có phản ánh sự khác biệt thực sự giữa các nhóm hay không, hay chúng chỉ là kết quả của sự ngẫu nhiên.

Có hai loại phân tích phương sai thông dụng nhất mà ta sẽ tìm hiểu sau đây:

- Phân tích phương sai 1 yếu tố (one-way ANOVA).
- Phân tích phương sai 2 yếu tố (two-way ANOVA).

2.2.1 Phân tích phương sai 1 yếu tố (one-way ANOVA)

Trong mô hình phân tích phương sai 1 yếu tố, chúng ta kiểm định so sánh trung bình của biến ngẫu nhiên X ở những tổng thể (còn gọi là nhóm) khác nhau dựa vào các mẫu quan sát lấy từ những tổng thể này. Các tổng thể được phân biệt bởi các mức độ khác nhau của yếu tố đang xem xét.

Các bước thực hiện mô hình phân tích phương sai một yếu tố:

- Đặt giả thiết bài toán
 - Các tổng thể có phân phối chuẩn:
 - Phương sai các tổng thể bằng nhau:
 - Các mẫu quan sát (từ các tổng thể) được lấy độc lập.

- Đặt giả thuyết kiểm định

Gọi $\mu_1, \mu_2, \dots, \mu_k$ lần lượt là trung bình tổng thể 1, 2, ..., k

Giả thuyết không H_0 :

$$\mu_1 = \mu_2 = \dots = \mu_k$$

Giả thuyết đối H_1 :

$$\exists \mu_i \neq \mu_j, \quad \text{với } i \neq j$$

- Tính giá trị kiểm định thống kê

Trong phân tích phương sai một yếu tố, giá trị kiểm định thống kê chính là giá trị F (-statistic). Giá trị F được sử dụng để kiểm tra xem có sự khác biệt đáng kể giữa các nhóm trong biến ngẫu nhiên X đang nghiên cứu hay không.

Công thức tính giá trị kiểm định thống kê F :

$$F_A = \frac{MSB}{MSW}$$

- Tìm miền bác bỏ

Miền bác bỏ trong phân tích phương sai một yếu tố nằm ở bên phải của phân phối F , được xác định dựa trên mức ý nghĩa thống kê cùng với bậc tự do của biến đổi giữa các nhóm (dfB) và biến đổi bên trong từng nhóm (dfW). Tức là:

$$RR = (f_\alpha(dfB; dfW); +\infty) = (f_\alpha(K-1; N-k); +\infty)$$

- Kết luận



Nếu giá trị thống kê kiểm định F thuộc vào miền bác bỏ RR, ta bác bỏ được giả thuyết H_0 , kết luận trung bình của các tổng thể là khác nhau (sự khác biệt có ý nghĩa thống kê). Ngược lại, ta chưa bác bỏ được H_0 , hay chưa có bằng chứng về sự khác biệt giữa trung bình của các tổng thể.

Sau khi ta bác bỏ được giả thuyết H_0 , thì với nhu cầu phân tích thêm sự khác biệt đến từ một(hay những)nhóm nào: nhóm nào đó có trung bình lớn hơn, bằng, hay nhỏ hơn so với những nhóm khác,...

Có nhiều phương pháp đưa đến kết quả mong muốn, ta còn gọi đó là các phương pháp so sánh bội (Multiple comparison methods). Trong đó có phương pháp dùng kiểm định LSD (Least Significant Difference Test).

Các bước tiến hành kiểm định LSD:

- Lần lượt kiểm định tất cả C_k^2 cặp trung bình μ_i và μ_j của hai nhóm khác nhau. Đặt giả thuyết:

Giả thuyết không H_0 :

$$\mu_i = \mu_j$$

Giả thuyết đối H_1 :

$$\mu_i \neq \mu_j$$

- Tính giá trị kiểm định thống kê

$$LSD_{i,j} = t_{\alpha/2}(N-k) \sqrt{MSW \left(\frac{1}{n_i} + \frac{1}{n_j} \right)}$$

- Kết luận

Nếu $|\bar{X}_i - \bar{X}_j| > LSD_{i,j}$, ta bác bỏ được giả thuyết, kết luận trung bình của hai nhóm H_0 là khác nhau (sự khác biệt có ý nghĩa thống kê). Ngược lại, ta chưa bác bỏ được, hay chưa có bằng chứng về sự khác biệt giữa trung bình của hai nhóm.

2.2.2 Phân tích phương sai 2 yếu tố (Two-way ANOVA)

ANOVA hai chiều là một mở rộng của ANOVA một chiều, cho phép kiểm tra sự ảnh hưởng đồng thời của hai yếu tố độc lập lên một biến phụ thuộc. Phương pháp này không chỉ xem xét ảnh hưởng chính của từng yếu tố mà còn kiểm tra sự tương tác giữa hai yếu tố đó.

Ví dụ: Bạn đang nghiên cứu về phương pháp giảng dạy và số giờ học ảnh hưởng đến kết quả học tập của học sinh. Bạn tiến hành thí nghiệm trong một trường học, chia các lớp học thành các nhóm khác nhau theo phương pháp giảng dạy (1 = phương pháp truyền thống, 2 = phương pháp sử dụng công nghệ, 3 = phương pháp kết hợp) và số giờ học mỗi tuần (1 = ít hơn 10 giờ, 2 = từ 10 đến 20 giờ). Sau một kỳ học, bạn đo điểm trung bình của các học sinh để đánh giá kết quả học tập.

Phân tích ANOVA hai chiều giúp bạn xác định xem phương pháp giảng dạy và số giờ học có ảnh hưởng đến kết quả học tập trung bình của học sinh hay không.

2.2.2.1 Phân loại:

Phân tích phương sai 2 yếu tố có thể được chia thành:

- **ANOVA 2 yếu tố không lặp:** Mỗi tổ hợp của hai yếu tố độc lập chỉ có một giá trị quan sát duy nhất cho biến phụ thuộc.
- **ANOVA 2 yếu tố có lặp:** Mỗi tổ hợp của hai yếu tố độc lập có nhiều giá trị quan sát cho biến phụ thuộc, điều này cho phép đo lặp lại theo thời gian hoặc điều kiện khác nhau.

Trong nghiên cứu này, chúng ta sẽ sử dụng **ANOVA 2 yếu tố không lặp** để phân tích dữ liệu.



2.2.2.2 Định nghĩa:

ANOVA hai chiều không lặp là một phương pháp phân tích trong đó mỗi tổ hợp của hai yếu tố độc lập chỉ có một giá trị quan sát duy nhất cho biến phụ thuộc. Điều này có nghĩa là không có sự lặp lại của các giá trị quan sát trong mỗi tổ hợp, do đó mô hình này phù hợp với dữ liệu mà mỗi cặp giá trị của các yếu tố độc lập chỉ được đo một lần.

$$Y_{ij} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \epsilon_{ij}$$

Trong đó:

- Y_{ij} là giá trị của biến phụ thuộc tại mức i của yếu tố A và mức j của yếu tố B.
- μ là giá trị trung bình tổng thể của dữ liệu.
- α_i là ảnh hưởng của mức i của yếu tố A.
- β_j là ảnh hưởng của mức j của yếu tố B.
- $(\alpha\beta)_{ij}$ là ảnh hưởng tương tác giữa yếu tố A và B.
- ϵ_{ij} là sai số ngẫu nhiên, được giả định là độc lập, có phân phối chuẩn với giá trị kỳ vọng bằng 0 và phương sai không đổi.

2.2.2.3 Điều kiện dữ liệu đầu vào:

Để áp dụng ANOVA hai chiều không lặp, dữ liệu phải thỏa mãn các điều kiện sau:

- (i) Biến phụ thuộc phải là biến liên tục (dạng khoảng, tỉ lệ).
- (ii) Có 2 biến độc lập là biến danh mục (từ 2 mức trở lên) và giá trị có thể dạng số hoặc chuỗi.
- (iii) Các quan sát trong mỗi nhóm và giữa các nhóm độc lập với nhau.
- (iv) Dữ liệu không có chứa các điểm dị biệt.
- (v) Dữ liệu của biến phụ thuộc ở mỗi nhóm phải có dạng phân phối chuẩn hoặc xấp xỉ phân phối chuẩn.
- (vi) Có sự đồng nhất về phương sai của biến phụ thuộc. Điều này có nghĩa là, phương sai của tổng thể trong mỗi nhóm đều bằng nhau.

2.2.2.4 Các bước tiến hành:

- 1) **Giả thuyết kiểm định:** Trong ANOVA hai chiều không lặp, chúng ta kiểm định các giả thuyết sau:

- **H0 cho yếu tố A:** Không có sự khác biệt có ý nghĩa thống kê về giá trị trung bình của biến phụ thuộc giữa các mức của yếu tố A.
- **H0 cho yếu tố B:** Không có sự khác biệt có ý nghĩa thống kê về giá trị trung bình của biến phụ thuộc giữa các mức của yếu tố B.
- **H0 cho tương tác giữa A và B:** Không có sự tương tác có ý nghĩa giữa yếu tố A và B.

- 2) **Kiểm định thống kê cho từng yếu tố:**

- **F-statistic cho yếu tố A:**

$$F_A = \frac{MS_{\text{betweenA}}}{MS_{\text{within}}}$$



- **Bậc tự do:** Bậc tự do giữa các mức của yếu tố A (df_{betweenA}):

$$df_{\text{betweenA}} = a - 1$$

với a là số mức của yếu tố A.

- **Xác định giá trị P:**

$$p_A = P(F > F_A \mid df_{\text{betweenA}}, df_{\text{within}})$$

→ Tương tự đối với yếu tố B.

3) Kiểm định thông kê cho tương tác giữa yếu tố A và B:

$$F_{\text{interaction}} = \frac{MS_{\text{interaction}}}{MS_{\text{within}}}$$

- 4) **Kết luận:** Nếu giá trị p của bất kỳ F-statistic nào nhỏ hơn mức ý nghĩa α đã chọn, chúng ta sẽ bác bỏ giả thuyết không (H_0). Giả thuyết không cho từng yếu tố (A, B, và tương tác) là không có sự ảnh hưởng có ý nghĩa thống kê từ yếu tố đó lên biến phụ thuộc. Điều này có nghĩa là, nếu p-value của F-statistic cho yếu tố A, B, hoặc sự tương tác giữa chúng nhỏ hơn α , chúng ta kết luận rằng yếu tố đó hoặc sự tương tác giữa các yếu tố có ảnh hưởng đáng kể đến biến phụ thuộc.

2.2.2.5 Ý nghĩa nghiên cứu:

ANOVA hai chiều giúp các nhà nghiên cứu hiểu rõ hơn về cách các yếu tố độc lập tương tác và ảnh hưởng đến biến phụ thuộc, từ đó có thể đưa ra các kết luận chính xác hơn về hiện tượng đang nghiên cứu.

2.3 Hồi quy tuyến tính đa biến

Hồi quy tuyến tính bội là phần mở rộng của hồi quy tuyến tính đơn. Nó được sử dụng khi chúng ta muốn dự đoán giá trị của một biến phản hồi dựa trên giá trị của hai hoặc nhiều biến giải thích. Hồi quy tuyến tính bội nghĩa là liên hệ biến phụ thuộc Y cho trước với nhiều biến độc lập $X_1, X_2, \dots, X_k$. Mô hình hồi quy tuyến tính bội có dạng tổng quát như sau:

$$Y = \beta_1 + \beta_2 X_2 + \beta_3 X_3 + \dots + \beta_n X_n$$

Trong đó: β_1 là hệ số tự do (hệ số chặn)

β_i : là hệ số quy hồi tuyến tính riêng (hay hệ số góc)

Y: là biến phụ thuộc

X_i : là biến độc lập

Hệ số $\beta_i > 0$: khi đó mối quan hệ giữa Y và X_i là thuận chiều

Hệ số $\beta_i < 0$: khi đó mối quan hệ giữa Y và X_i là ngược chiều

Hệ số $\beta_i = 0$ nghĩa là Y và X_i không có tương quan với nhau

Hồi quy tuyến tính bội là phương pháp để dự đoán giá trị biến phụ thuộc (Y) dựa trên giá trị của biến độc lập (X).

Hồi quy mẫu

Ta biết rằng dù mô hình có nhiều biến độc lập nhưng vẫn tồn tại những yếu tố tác động đến biến phụ thuộc nhưng không đưa vào mô hình vì nhiều lý do.

Do đó trong mô hình vẫn tồn tại sai số ngẫu nhiên u đại diện cho các yếu tố khác ngoài biến $X_j (j = 2, 3, \dots, k)$ có tác động đến Y nhưng không đưa vào mô hình như là biến số.

Xét một mẫu ngẫu nhiên với n quan sát cụ thể, ta có hồi quy mẫu như sau:

$$\hat{Y}_i = \hat{\beta}_1 + \hat{\beta}_2 \hat{X}_{2i} + \hat{\beta}_3 \hat{X}_{3i} + \dots + \hat{\beta}_k \hat{X}_{ki} + e_i$$



Với e_i là phần dư quan sát, được tính bởi công thức: $e_i = Y_i - \hat{Y}_i$

Các giả thiết của mô hình hồi qui bội:

- Việc ước lượng được dựa trên cơ sở ngẫu nhiên.
- Kỳ vọng của sai số ngẫu nhiên tại mỗi giá trị bằng 0.
- Phương sai của sai số ngẫu nhiên tại các giá trị () đều bằng nhau.
- Giữa các biến độc lập không có cộng tuyến tính hoàn hảo, nghĩa là không tồn tại hằng số không đồng thời bằng 0.

⇒ Từ giả 3 có thể nói sai số ngẫu nhiên tuân theo phân phối chuẩn.

⇒ Giả thiết 4 được đưa ra nhằm loại trừ tình huống suy ra các biến còn lại khi có ít nhất một trong các biến vì $X_i (i = 2, 3, \dots, k)$ có quan hệ cộng tuyến tính hoàn hảo.

3 Tiền xử lý số liệu.

3.1 Đọc dữ liệu.

```
readFile <- read.csv("All_GPUs.csv")
```

Để đọc dữ liệu chứa trong tập tin **All_GPUs.csv**, ta sử dụng lệnh `read.csv()` để đọc dữ liệu từ tập tin CSV. Dữ liệu đọc được từ tập tin sẽ được lưu vào biến `readFile`.

```
head(readFile)
```

```
View(readFile)
```

Để kiểm tra dữ liệu chứa trong biến `readFile`, ta có thể sử dụng lệnh `head()` để xem một số dòng đầu của tập tin; lệnh `View()` để xem tổng quan bảng dữ liệu đã nhập.

```
> readFile <- read.csv("C:/Users/dangk/Downloads/All_GPUs.csv")
> head(readFile)
  Architecture Best_Resolution Boost_Clock Core_Speed DTL_Connection Dedicated Direct_X DisplayPort_Connection HDMI_Connection
1 Tesla G90b 1366 x 768 738 MHz 2 Yes DX 10.0 NA 0
2 R600 XT 1366 x 768 738 MHz 2 Yes DX 10 NA 0
3 R600 PRO 1366 x 768 738 MHz 2 Yes DX 10 NA 0
4 RV630 1024 x 768 738 MHz 2 Yes DX 10 NA 0
5 RV630 1024 x 768 738 MHz 2 Yes DX 10 NA 0
6 RV630 1024 x 768 738 MHz 2 Yes DX 10 NA 0
  Integrated_L2_Cache Manufacturer Max_Power Memory Memory_Bandwidth Memory_Bus Memory_Speed Memory_Type
1 No 0KB Nvidia 141 Watts 1024 MB 64GB/sec 256 Bit 1000 MHz GDDR3
2 No 0KB AMD 215 Watts 512 MB 10GB/sec 256 Bit 800 MHz GDDR3
3 No 0KB AMD 200 Watts 512 MB 11.2GB/sec 256 Bit 800 MHz GDDR3
4 No 0KB AMD 256 MB 16.8GB/sec 128 Bit 1150 MHz GDDR4
5 No 0KB AMD 45 Watts 256 MB 22.4GB/sec 128 Bit 700 MHz GDDR3
6 No 0KB AMD 50 Watts 256 MB 15.2GB/sec 128 Bit 1100 MHz GDDR4
  Name Notebook_GPU Open_GL PSU Pixel_Rate Power_Connector Process ROPS
1 GeForce GTS 150 No 3.3 450 watt & 38 Amps 12 GPixel/s None 55nm 16
2 Radeon HD 2900 XT 512MB No 3.1 550 watt & 35 Amps 12 GPixel/s None 80nm 16
3 Radeon HD 2900 Pro No 3.1 550 watt & 35 Amps 10 GPixel/s None 80nm 16
4 Radeon HD 2600 XT Diamond Edition No 3.3 3 GPixel/s None 65nm 4
5 Radeon HD 2600 XT No 3.1 400 watt & 25 Amps 3 GPixel/s None 65nm 4
6 Radeon HD 2600 XT 256MB GDDR4 No 3.3 400 watt & 26 Amps 3 GPixel/s None 65nm 4
  Release_Date Release_Price Resolution_Width SLT_CrossFire Shader TMUs Texture_Rate VGA_Connection
1 v01-Mar-2009 2560x1600 Yes 4 64 47 GTexel/s 0
2 v14-May-2007 2560x1600 Yes 4 16 12 GTexel/s 0
3 v07-Dec-2007 2560x1600 Yes 4 16 10 GTexel/s 0
4 v01-Jul-2007 2560x1600 Yes 4 8 7 GTexel/s 0
5 v28-Jun-2007 2560x1600 Yes 4 8 6 GTexel/s 0
6 v26-Jun-2007 2560x1600 Yes 4 8 6 GTexel/s 0
```

Hình 1: Lệnh `Head()` dùng để xem một phần của bảng dữ liệu.



#	Architecture	Best_Resolution	Boost_Clock	Core_Speed	DVI_Connection	Dedicated	Direct_X	DisplayPort_Connection	HDMI_Connection	Integrated
1	Fermi G210			738 MHz		2 Yes	DX 10.1	No		0 No
2	Radeon XT	1366 x 768		--		2 Yes	DX 10	No		0 No
3	Radeon R420	1366 x 768		--		2 Yes	DX 10	No		0 No
4	R420	1024 x 768		--		2 Yes	DX 10	No		0 No
5	R420	1024 x 768		--		2 Yes	DX 10	No		0 No
6	R420	1024 x 768		--		2 Yes	DX 10	No		0 No
7	ATI RV790 XT	1280 x 1024		870 MHz		1 Yes	DX 12.1	No		1 No
8	Radeon GT	1024 x 768		--		2 Yes	DX 10	No		0 No
9	Radeon XT GL	1280 x 1024		--		0 Yes	DX 11.2	No		0 No
10	R410			--	No	No	DX 7	No	No	0 No
11	NAVIO2L A2			--		2 No	DX 8.1	No		0 No
12	Fermi GF110	1280 x 1024		650 MHz		2 Yes	DX 12.0			1 No
13	Kaplan GA110			715 MHz		0 Yes	DX 12.0	No		0 No
14	Kaplan GA110	2560 x 1600		715 MHz		0 Yes	DX 12.0	No		0 No
15	R420			--	No	No	DX 7	No	No	0 No
16	GCN 1.1 Oland XT + Kaveri	1280 x 800	1100 MHz	1050 MHz		1 Yes	DX 12.0			1 No
17	Kaplan GA104 a2			--		0 Yes	DX 11.1	No		0 No
18	Kaplan GA110			712 MHz		0 Yes	DX 12.0	No		0 No
19	Skyline-GT3-E	1366 x 768	1800 MHz	180 MHz	No	No	DX 12.1	No	No	Yes
20	R580	1024 x 768		--		1 Yes	DX 9			0 No
21	Fermi GF100			575 MHz		1 Yes	DX 12.0	No		0 No
22	Fermi GF100			575 MHz		1 Yes	DX 12.0	No		0 No
23	NAVIO A2			--		2 Yes	DX 8.1	No		0 No
24	NAVIOGL A2			--		2 Yes	DX 8.1	No		0 No
25	Sivision GT3 B	1366 x 768	1800 MHz	200 MHz	No	No	DX 12.1	No	No	Yes

Hình 2: Lệnh View() dùng để xem toàn bộ bảng dữ liệu.

3.2 Trích xuất dữ liệu.

Từ 34 thuộc tính có trong tập dữ liệu, ta chọn ra 10 thuộc tính quan trọng của GPU để xử lý: Name, Best_Resolution, Core_Speed, Max_Power, Memory, Memory_Bandwidth, Memory_Speed, Manufacturer, Release_Date, Release_Price

```
data <- readfile[,c("Name",  
                    "Best_Resolution",  
                    "Core_Speed",  
                    "Max_Power",  
                    "Memory",  
                    "Memory_Bandwidth",  
                    "Memory_Speed",  
                    "Manufacturer",  
                    "Release_Date",  
                    "Release_Price")]
```

Ta sẽ trích xuất dữ liệu của các cột có tên tương ứng vào trong 1 biến data bằng cách sử dụng toán tử [] để truy cập và lấy ra một phần của tập tin dữ liệu. Ở đây, chúng ta sẽ lấy tất cả các dòng của các cột có tên tương ứng. Lệnh c() được sử dụng để chỉ định các cột cụ thể cần trích xuất và gán giá trị đó cho biến data thông qua toán tử <-. Chúng ta có thể sử dụng lệnh View(data) để xem toàn bộ dữ liệu chứa trong data.



#	Name	Best Resolution	Core Speed	Max Power	Memory	Memory Bandwidth	Memory Speed	Manufacturer	Release Date
1	Geforce GT5 150	1280 x 768	730 MHz	141 Watts	1024 MB	64GB/sec	1000 MHz	Nvidia	01-Mar-2009
2	Radeon HD 2300 XT 512MB	1280 x 768	-	215 Watts	512 MB	10GB/sec	825 MHz	AMD	14-May-2007
3	Radeon HD 2300 Pro	1280 x 768	-	200 Watts	512 MB	51.2GB/sec	800 MHz	AMD	07-Dec-2007
4	Radeon HD 2300 XT Diamond Edition	1024 x 768	-	-	256 MB	36.8GB/sec	1150 MHz	AMD	01-Jul-2007
5	Radeon HD 2300 XT	1024 x 768	-	45 Watts	256 MB	22.4GB/sec	700 MHz	AMD	25-Jun-2007
6	Radeon HD 2300 XT 256MB GDDR4	1024 x 768	-	50 Watts	256 MB	35.2GB/sec	1100 MHz	AMD	25-Jun-2007
7	Radeon HD 4390 Sapphire Vapor-X OC 2GB Edition	1920 x 1080	870 MHz	190 Watts	2048 MB	134.4GB/sec	1050 MHz	AMD	13-Jul-2009
8	Radeon HD 2300 GT	1024 x 768	-	150 Watts	256 MB	51.2GB/sec	800 MHz	AMD	04-Nov-2007
9	FirePro 2300	1920 x 1080	-	150 Watts	2048 MB	109GB/sec	1250 MHz	AMD	15-Jan-2014
10	Radeon 7000 96MB	-	-	32 Watts	64 MB	2.5GB/sec	280 MHz	AMD	02-Jan-2000
11	Quadro 980 XGL	-	-	-	128 MB	5.2GB/sec	325 MHz	Nvidia	01-Nov-2000
12	Tesla M2050	1920 x 1080	660 MHz	250 Watts	6144 MB	177.6GB/sec	925 MHz	Nvidia	25-Jul-2011
13	Tesla K20	-	705 MHz	225 Watts	5120 MB	168GB/sec	1050 MHz	Nvidia	01-Nov-2012
14	Tesla K40c	2560 x 1600	705 MHz	245 Watts	12288 MB	260.4GB/sec	1500 MHz	Nvidia	12-Nov-2013
15	All-in-Wonder Radeon 7500	-	-	-	64 MB	5.8GB/sec	380 MHz	AMD	22-Jan-2000
16	Radeon R7 220 2GB NGI OC 2GB + Radeon R7 7070K Dual	1680 x 900	1150 MHz	150 Watts	2072 MB	57.6GB/sec	900 MHz	AMD	20-May-2013
17	Tesla K10	-	-	380 Watts	8192 MB	32GB/sec	1350 MHz	Nvidia	15-Mar-2010
18	Tesla K20X	-	732 MHz	235 Watts	6144 MB	149.9GB/sec	1300 MHz	Nvidia	12-Nov-2013
19	Ins i3 E717U	1280 x 768	280 MHz	28 Watts	28 Watts	34.7GB/sec	1051 MHz	Intel	01-Sep-2007
20	Radeon 9800 XT	1024 x 768	-	65 Watts	256 MB	23.4GB/sec	750 MHz	AMD	01-Oct-2007
21	Tesla C2070	-	575 MHz	230 Watts	8168 MB	168GB/sec	750 MHz	Nvidia	01-Sep-2007
22	Tesla C2075	-	575 MHz	247 Watts	6144 MB	168GB/sec	750 MHz	Nvidia	01-Jul-2007
23	Quadro FX 580	-	-	-	128 MB	5.6GB/sec	325 MHz	Nvidia	19-Feb-2000

Hình 3: Bảng dữ liệu sau khi đã được trích xuất.

3.3 Xử lý định dạng dữ liệu.

Ta sẽ sử dụng thư viện **tidyr** có sẵn trên R Studio để thực hiện các bước tính toán sơ bộ, tách đơn vị của các mẫu định lượng.

```
library(tidyr)
```

Đối với mẫu định tính **Name**: Do chỉ quan tâm đến thương hiệu GPU và bỏ qua các thông tin phụ có trong tên, nên ta sẽ chỉ giữ lại từ đầu trong các phần tử của mẫu.

```
data$Name <- sapply(strsplit(data$Name, " "), "[", 1)
```

Chúng ta sẽ sử dụng toán tử **\$** để truy cập vào các thành phần của data, cụ thể chúng ta sẽ truy cập vào cột **Name**. Hàm **strsplit** sẽ tách chuỗi ký tự thành vector chứa các phần tử với dấu cách " " là dấu phân cách. Sau đó, ta sử dụng hàm **sapply()** áp dụng toán tử truy cập và lấy phần tử đầu tiên của mỗi vector, sau đó gán trở lại cột **Name** của data.

#	Name	Best Resolution	Core Speed	Max Power	Memory	Memory Bandwidth	Memory Speed	Manufacturer	Release Date	Release Price
1	Geforce	1280 x 768	730 MHz	141 Watts	1024 MB	64GB/sec	1000 MHz	Nvidia	01-Mar-2009	
2	Radeon	1280 x 768	-	215 Watts	512 MB	10GB/sec	825 MHz	AMD	14-May-2007	
3	Radeon	1280 x 768	-	200 Watts	512 MB	51.2GB/sec	800 MHz	AMD	07-Dec-2007	
4	Radeon	1024 x 768	-	-	256 MB	36.8GB/sec	1150 MHz	AMD	01-Jul-2007	
5	Radeon	1024 x 768	-	45 Watts	256 MB	22.4GB/sec	700 MHz	AMD	25-Jun-2007	
6	Radeon	1024 x 768	-	50 Watts	256 MB	35.2GB/sec	1100 MHz	AMD	25-Jun-2007	
7	Radeon	1920 x 1080	870 MHz	190 Watts	2048 MB	134.4GB/sec	1050 MHz	AMD	13-Jul-2009	
8	Radeon	1024 x 768	-	150 Watts	256 MB	51.2GB/sec	800 MHz	AMD	06-Nov-2007	
9	FirePro	1920 x 1080	-	150 Watts	2048 MB	109GB/sec	1250 MHz	AMD	15-Jan-2014	
10	Radeon	-	-	32 Watts	64 MB	2.5GB/sec	366 MHz	AMD	02-Jan-2000	
11	Quadro	-	-	-	128 MB	5.2GB/sec	325 MHz	Nvidia	01-Nov-2000	
12	Tesla	1920 x 1080	660 MHz	250 Watts	6144 MB	177.6GB/sec	925 MHz	Nvidia	25-Jul-2011	
13	Tesla	-	705 MHz	225 Watts	5120 MB	168GB/sec	1050 MHz	Nvidia	01-Nov-2012	
14	Tesla	2560 x 1600	705 MHz	245 Watts	12288 MB	260.4GB/sec	1500 MHz	Nvidia	12-Nov-2013	

Hình 4: Lấy chữ cái đầu tiên của cột Name.

Đối với mẫu **Best_Resolution**: Chúng ta sẽ tính số điểm ảnh (pixels) mà GPU hoạt động tốt nhất bằng cách thực hiện phép nhân số giữa pixels theo chiều ngang và dọc. Sau đó, chúng ta sẽ đổi tên cột này thành **Number_of_Pixels**

```
data$Best_Resolution <- sapply(strsplit(data$Best_Resolution, "x"), function(x)
  as.numeric(x[1]) * as.numeric(x[2]))
names(data)[names(data) == "Best_Resolution"] <- "Number_of_Pixels"
```

Việc thực hiện phép nhân (*) giữa chiều dài và chiều rộng cần giá trị của chiều dài và chiều rộng cần phải ở định dạng số. Do đó chúng ta cần phải thực hiện các bước xử lý trung gian trước khi thực hiện phép nhân.

Trước tiên, chúng ta truy cập vào cột **Best_Resolution** bằng toán tử \$, sử dụng hàm `strsplit()` để tách thành các phần tử với dấu "x" là dấu phân cách. Tiếp đó, chúng ta sẽ định nghĩa 1 hàm nhận một vector x. Hàm này sẽ chuyển định dạng của phần tử thứ nhất (x[1]) và phần tử thứ hai (x[2]) về định dạng số bằng hàm `as.numeric()` sau đó thực hiện phép nhân. Sử dụng hàm `sapply()` để áp dụng hàm trên lên từng phần tử vector được chia bởi hàm `strsplit()`. Kết quả sẽ được gán vào cột **Best_Resolution**.

Sau khi thực hiện phép tính, chúng ta sẽ đổi tên cột **Best_Resolution** thành **Number_of_Pixels** bằng cách truy cập vào cột có tên là **Best_Resolution** sau đó gán giá trị tên mới là **Number_of_Pixels**.



Name	Number_of_Pixels	Core_Speed	Max_Power	Memory	Memory_Bandwidth	Memory_Speed	Manufacturer	Release_Date	Release_Price
1 GeForce	N/A	730 MHz	141 Watts	1024 MB	64GB/sec	9000 MHz	Nvidia	01-Mar-2009	
2 Radeon	1048988	-	215 Watts	512 MB	106GB/sec	830 MHz	AMD	14-May-2007	
3 Radeon	1048988	-	206 Watts	512 MB	51.2GB/sec	800 MHz	AMD	07-Dec-2007	
4 Radeon	786432	-	-	256 MB	36.8GB/sec	1150 MHz	AMD	01-Jun-2007	
5 Radeon	786432	-	45 Watts	256 MB	22.4GB/sec	700 MHz	AMD	28-Jun-2007	
6 Radeon	786432	-	60 Watts	256 MB	36.2GB/sec	1100 MHz	AMD	24-Jun-2007	
7 Radeon	2073600	870 MHz	190 Watts	2048 MB	134.8GB/sec	9050 MHz	AMD	13-Jul-2009	
8 Radeon	786432	-	150 Watts	256 MB	51.2GB/sec	800 MHz	AMD	06-Nov-2007	
9 FirePro	2073600	-	150 Watts	2048 MB	100GB/sec	1250 MHz	AMD	18-Jan-2014	
10 Radeon	N/A	-	32 Watts	64 MB	2.9GB/sec	366 MHz	AMD	03-Jan-2001	
11 Quadro	N/A	-	-	128 MB	5.2GB/sec	525 MHz	Nvidia	01-Nov-2002	
12 Tesla	2073600	890 MHz	250 Watts	6144 MB	177.8GB/sec	925 MHz	Nvidia	25-Jul-2011	
13 Tesla	N/A	755 MHz	225 Watts	5120 MB	100GB/sec	9050 MHz	Nvidia	01-Nov-2012	
14 Tesla	4096000	706 MHz	245 Watts	12288 MB	288.8GB/sec	1502 MHz	Nvidia	12-Nov-2013	

Hình 5: Thực hiện tính toán và đổi tên cột **Best_Resolution** thành **Number_of_Pixels**

Đối với các cột **Core_Speed**, **Max_Power**, **Memory**, **Memory_Bandwidth**, **Memory_Speed**: Chúng ta sẽ thực hiện tách thành hai cột chứa giá trị và đơn vị. Cột giá trị sẽ được chuyển thành định dạng số.

Do cột **Core_Speed** có chứa các giá trị khuyết thiếu được biểu diễn bởi "-" nên cần được loại bỏ trước khi xử lý:

```
data$Core_Speed[grepl("-", data$Core_Speed)] <- ""
```

Chúng ta sẽ tiến hành truy cập vào cột **Core_Speed** trong tập dữ liệu data, dùng hàm `grepl()` để kiểm tra mỗi giá trị trong cột **Core_Speed** có chứa dấu "-" hay không. Nếu có, lấy giá trị trống "" gán vào vị trí các giá trị "-".



Name	Number_of_Pixels	Core_Speed	Max_Power	Memory	Memory_Bandwidth	Memory_Speed	Manufacturer	Release_Date	Release_Price
1 GeForce	N/A	730 MHz	141 Watts	1024 MB	64GB/sec	9000 MHz	Nvidia	01-Mar-2009	
2 Radeon	1048988	-	215 Watts	512 MB	106GB/sec	830 MHz	AMD	14-May-2007	
3 Radeon	1048988	-	206 Watts	512 MB	51.2GB/sec	800 MHz	AMD	07-Dec-2007	
4 Radeon	786432	-	-	256 MB	36.8GB/sec	1150 MHz	AMD	01-Jun-2007	
5 Radeon	786432	-	45 Watts	256 MB	22.4GB/sec	700 MHz	AMD	28-Jun-2007	
6 Radeon	786432	-	60 Watts	256 MB	36.2GB/sec	1100 MHz	AMD	24-Jun-2007	
7 Radeon	2073600	870 MHz	190 Watts	2048 MB	134.8GB/sec	9050 MHz	AMD	13-Jul-2009	
8 Radeon	786432	-	150 Watts	256 MB	51.2GB/sec	800 MHz	AMD	06-Nov-2007	
9 FirePro	2073600	-	150 Watts	2048 MB	100GB/sec	1250 MHz	AMD	18-Jan-2014	
10 Radeon	N/A	-	32 Watts	64 MB	2.9GB/sec	366 MHz	AMD	03-Jan-2001	
11 Quadro	N/A	-	-	128 MB	5.2GB/sec	525 MHz	Nvidia	01-Nov-2002	
12 Tesla	2073600	890 MHz	250 Watts	6144 MB	177.8GB/sec	925 MHz	Nvidia	25-Jul-2011	
13 Tesla	N/A	755 MHz	225 Watts	5120 MB	100GB/sec	9050 MHz	Nvidia	01-Nov-2012	
14 Tesla	4096000	706 MHz	245 Watts	12288 MB	288.8GB/sec	1502 MHz	Nvidia	12-Nov-2013	

Hình 6: Thay thế các giá trị "-" có trong **Core_Speed** bằng ""

Sau khi đã loại bỏ các giá trị "-" chứa trong cột **Core_Speed**, chúng ta sẽ tiến hành tách các cột thành 2 cột chứa giá trị và đơn vị.



```
data <- separate(data, col= Core_Speed, into =  
  c("Core_Speed_Value", "Core_Speed_Unit"), sep = " ", fill = "right")  
data$Core_Speed_Value <- as.numeric(data$Core_Speed_Value)  
  
data <- separate(data, col= Max_Power, into =  
  c("Max_Power_Value", "Max_Power_Unit"), sep = " ", fill = "right")  
data$Max_Power_Value <- as.numeric(data$Max_Power_Value)  
  
data <- separate(data, col= Memory, into = c("Memory_Value", "Memory_Unit"),  
  sep = " ", fill = "right", extra = "drop")  
data$Memory_Value <- as.numeric(data$Memory_Value)  
  
data <- separate(data, col= Memory_Bandwidth, into =  
  c("Memory_Bandwidth_Value", "Memory_Bandwidth_Unit"), sep =  
  "(?<=\\d) (?=[A-Za-z])", fill = "right")  
data$Memory_Bandwidth_Value <- as.numeric(data$Memory_Bandwidth_Value)  
  
data <- separate(data, col= Memory_Speed, into =  
  c("Memory_Speed_Value", "Memory_Speed_Unit"), sep = " ", fill = "right")  
data$Memory_Speed_Value <- as.numeric(data$Memory_Speed_Value)
```

Chúng ta sẽ sử dụng hàm `separate()` để chia tách các cột thành nhiều cột mới. Trước tiên, chúng ta sẽ tiến hành chia cột **Core_Speed** trong `data` thành 2 cột **Core_Speed_Value** và **Core_Speed_Unit** để chứa lần lượt giá trị và đơn vị của cột **Core_Speed**. Chỉ định dấu cách " " để chia tách cột, và chúng ta sẽ điền từ bên phải khi có các giá trị còn thiếu. Cột **Core_Speed_Value** sẽ được chuyển về định dạng số bằng hàm `as.numeric()`. Làm tương tự với các cột **Max_Power**, **Memory**, **Memory_Bandwidth**, **Memory_Speed**.

Trong trường hợp của cột **Memory_Bandwidth**, các ký tự chữ và số được viết liền nhau, ta không thể sử dụng dấu cách " " làm dấu phân cách giữa các cột. Vì vậy, chúng ta cần 1 biểu thức để xác định điểm phân cách, cụ thể là vị trí mà ngay sau một chữ số và trước một chữ cái. `(?<=\\d)` sẽ kiểm tra xem trước vị trí hiện tại có phải là một chữ số hay không, `(?=[A-Za-z])` sẽ kiểm tra vị trí ngay sau có phải là một ký tự hay không. Khi kết hợp cả hai điều kiện này sẽ cho ra vị trí phân tách giữa phần giá trị và đơn vị trong cột **Memory_Bandwidth**.

Khi thực hiện chia tách cột **Memory**, ta nhận thấy, có những ô có giá trị không xác định (NA), chúng ta cần có thêm một tham số **Extra** để loại bỏ các phần thừa.

#	Name	Number_of_Pixels	Core_Speed_Value	Core_Speed_Unit	Max_Power_Value	Max_Power_Unit	Memory_Value	Memory_Unit	Memory_Bandwidth_Value
1	GeForce	104	736	MHz	127	Watts	1024	MB	44
2	Radeon	1049088	640	Hz	275	Watts	512	MB	105
3	Radeon	1049088	640	Hz	200	Watts	512	MB	51
4	Radeon	765432	640	Hz	144	Watts	256	MB	35
5	Radeon	765432	640	Hz	65	Watts	256	MB	23
6	Radeon	765432	640	Hz	50	Watts	256	MB	35
7	Radeon	2073908	870	MHz	100	Watts	2048	MB	134
8	Radeon	765432	640	Hz	100	Watts	256	MB	51
9	Radeon	2073908	640	Hz	160	Watts	2048	MB	145
10	Radeon	104	640	Hz	32	Watts	64	MB	2
11	Quadro	104	640	Hz	140	Watts	512	MB	8
12	Tesla	2073908	630	MHz	250	Watts	6144	MB	171
13	Tesla	104	705	MHz	225	Watts	5120	MB	148
14	Tesla	4762104	714	MHz	145	Watts	4736	MB	144

Hình 7: Bảng dữ liệu sau khi thực hiện chia tách.

```
table(data$Core_Speed_Unit)  
table(data$Max_Power_Unit)  
table(data$Memory_Unit)  
table(data$Memory_Bandwidth_Unit)  
table(data$Memory_Speed_Unit)
```



Chúng ta sẽ thực hiện kiểm tra đơn vị của các cột bằng hàm `table()`. Hàm `table()` có tác dụng tính toán và hiển thị bảng tần suất của các giá trị.

```
> table(data$Core_Speed_Unit)

MHz
2470
> table(data$Max_Power_Unit)

watts
2781
> table(data$Memory_Unit)

MB
2986
> table(data$Memory_Bandwidth_Unit)

GB/sec MB/sec
3281      4
> table(data$Memory_Speed_Unit)

MHZ
3301
```

Hình 8: Sử dụng hàm `table()` để kiểm tra đơn vị.

Có thể thấy rằng, các cột **Core_Speed**, **Max_Power**, **Memory** và **Memory_Speed** đều có cùng một loại đơn vị. Tuy nhiên, ở **Memory_Bandwidth** có 2 loại đơn vị khác nhau - **[GB/sec]** và **[MB/sec]**. Vậy nên, chúng ta sẽ chuyển các giá trị có đơn vị **[MB/sec]** về **[GB/sec]** bằng cách thực hiện phép chia. ($1[\text{MB/sec}] = \frac{1}{1024}[\text{GB/sec}]$)

```
data$Memory_Bandwidth_Value <- ifelse(data$Memory_Bandwidth_Unit ==
  "MB/sec", data$Memory_Bandwidth_Value/1024, data$Memory_Bandwidth_Value)
data$Memory_Bandwidth_Unit <- ifelse(data$Memory_Bandwidth_Unit ==
  "MB/sec", "GB/sec", data$Memory_Bandwidth_Unit)
```

Chúng ta sẽ sử dụng hàm `ifelse` để kiểm tra đơn vị. Đối với các cột có đơn vị là **[MB/sec]**, chúng ta sẽ thực hiện phép chia cho 1024 đối với cột **Memory_Bandwidth_Value** và chuyển đổi đơn vị trong cột **Memory_Bandwidth_Unit** thành **[MB/sec]** thành **[GB/sec]**.

```
> table(data$Memory_Bandwidth_Unit)

GB/sec
3285
```

Hình 9: Thực hiện chuyển đổi đơn vị trong cột **Memory_Bandwidth**.

```
data$Release_Date <- as.Date(sub("^\\s*\\n", "", data$Release_Date
), format="%d-%b-%Y")
data$Release_Date <- as.numeric(format(data$Release_Date, "%Y")) +
  as.numeric(format(data$Release_Date, "%m"))/12
```



Đối với **Release_Date**: Chuyển về định dạng ngày tháng năm bằng lệnh `as.Date()`. Trước khi chuyển đổi, chúng ta sẽ loại bỏ tất cả các khoảng trắng và ký tự xuống dòng trong chuỗi ngày tháng. Điều này đảm bảo rằng chuỗi ngày tháng có định dạng đúng. Chúng ta sẽ chuyển đổi sẽ có dạng là ngày - tháng 3 chữ cái - năm (ví dụ: "04 - Aug - 2013"). Sau khi đã định dạng, chúng ta sẽ chuyển đổi sang giá trị số bằng công thức:

$$\text{Release_Date} = \text{Năm} + \frac{1}{12} \text{Tháng}$$

Memory_Value	Memory_Unit	Memory_Bandwidth_Value	Memory_Bandwidth_Unit	Memory_Speed_Value	Memory_Speed_Unit	Manufacturer	Release_Date
1524	MB	640	GB/sec	1080	MHz	Nvidia	2009.250
512	MB	160.0	GB/sec	825	MHz	AMD	2007.417
512	MB	51.2	GB/sec	880	MHz	AMD	2008.000
256	MB	36.8	GB/sec	1150	MHz	AMD	2007.583
256	MB	22.4	GB/sec	780	MHz	AMD	2007.500
256	MB	35.2	GB/sec	1180	MHz	AMD	2007.500
2648	MB	1344	GB/sec	1050	MHz	AMD	2009.583
256	MB	51.2	GB/sec	880	MHz	AMD	2007.517
2648	MB	160.0	GB/sec	1250	MHz	AMD	2014.083
64	MB	2.9	GB/sec	246	MHz	AMD	2004.083
128	MB	5.2	GB/sec	325	MHz	Nvidia	2002.917
6144	MB	177.6	GB/sec	925	MHz	Nvidia	2011.583
5120	MB	148.0	GB/sec	1050	MHz	Nvidia	2012.917

Hình 10: Chuyển đổi **Release_Date** về định dạng số.

```
data$Release_Price <- gsub("$", "", data$Release_Price)
data$Release_Price <- as.numeric(data$Release_Price)
```

Đối với **Release_Price**: Chúng ta sẽ thực hiện xóa bỏ đơn vị và chuyển sang định dạng số. Để xóa ký tự "\$" có trong chuỗi, chúng ta sẽ sử dụng hàm `gsub()` để xóa. Sau đó sử dụng hàm `as.numeric()` để chuyển về định dạng số.

Memory_Unit	Memory_Bandwidth_Value	Memory_Bandwidth_Unit	Memory_Speed_Value	Memory_Speed_Unit	Manufacturer	Release_Date	Release_Price
MB	29.9	GB/sec	825	MHz	Intel	2015.500	149
MB	29.9	GB/sec	825	MHz	Intel	2015.500	149
MB	547.2	GB/sec	1425	MHz	Nvidia	2013.333	1999.30
MB	285.4	GB/sec	1532	MHz	Nvidia	2013.167	149
MB	672.0	GB/sec	1750	MHz	Nvidia	2014.417	2899.30
MB	672.0	GB/sec	1750	MHz	Nvidia	2014.417	149

Hình 11: Chuyển đổi định dạng số cho cột **Release_Price**

```
str(data)
View(data)
```

Chúng ta sẽ dùng hàm `str()` để kiểm tra kiểu dữ liệu của các cột, hàm `View()` để xem tổng thể bảng dữ liệu sau định dạng.

#	Name	Number_of_Pins	Core_Speed_Value	Core_Speed_Unit	Max_Power_Value	Max_Power_Unit	Memory_Value	Memory_Unit	Memory_Bandwidth_Value
1	GeForce	NA	736	MHz	121	Watts	1524	MB	64
2	Radeon	1048088	NA	NA	215	Watts	512	MB	160
3	Radeon	1048088	NA	NA	200	Watts	512	MB	51
4	Radeon	706432	NA	NA	NA	NA	256	MB	36
5	Radeon	706432	NA	NA	45	Watts	256	MB	22
6	Radeon	706432	NA	NA	50	Watts	256	MB	35
7	Radeon	2073688	870	MHz	190	Watts	2648	MB	1344
8	Radeon	706432	NA	NA	950	Watts	352	MB	51
9	FuryPro	2073688	NA	NA	150	Watts	2648	MB	160
10	Radeon	NA	NA	NA	32	Watts	64	MB	2
11	Quadro	NA	NA	NA	NA	NA	128	MB	5
12	Tesla	2073688	650	MHz	250	Watts	6144	MB	177
13	Tesla	NA	795	MHz	225	Watts	5120	MB	148
14	Tesla	4000000	NA	NA	NA	NA	1776	MB	177

Hình 12: Tổng thể bảng dữ liệu sau định dạng.

```
> str(data)
'data.frame': 3406 obs. of 15 variables:
 $ Name      : chr "GeForce" "Radeon" "Radeon" "Radeon" ...
 $ Number_of_Pixels : num NA 1049088 1049088 786432 786432 ...
 $ Core_Speed_Value : num 738 NA NA NA NA NA 870 NA NA NA ...
 $ Core_Speed_Unit  : chr "MHz" NA NA NA ...
 $ Max_Power_Value  : num 141 215 200 NA 45 50 190 150 150 32 ...
 $ Max_Power_Unit   : chr "Watts" "Watts" "Watts" NA ...
 $ Memory_Value     : num 1024 512 512 256 256 ...
 $ Memory_Unit      : chr "MB" "MB" "MB" "MB" ...
 $ Memory_Bandwidth_Value: num 64 106 51.2 36.8 22.4 ...
 $ Memory_Bandwidth_Unit: chr "GB/sec" "GB/sec" "GB/sec" "GB/sec" ...
 $ Memory_Speed_Value : num 1000 828 800 1150 700 1100 1050 800 1250 366 ...
 $ Memory_Speed_Unit  : chr "MHz" "MHz" "MHz" "MHz" ...
 $ Manufacturer     : chr "Nvidia" "AMD" "AMD" "AMD" ...
 $ Release_Date      : num 2009 2007 2008 2008 2008 ...
 $ Release_Price     : num NA NA NA NA NA NA NA NA NA ...
```

Hình 13: Kiểu dữ liệu của các cột trong Data Frame

3.4 Xử lý dữ liệu khuyết thiếu.

Với dữ liệu bảng, việc các trường thông tin bị khuyết (có giá trị NA) là thường xuyên xảy ra. Việc này đến từ quá trình thu thập dữ liệu. Người dùng có thể thực sự không có thông tin đó hoặc không muốn tiết lộ vì lý do cá nhân.

Có nhiều cách để xử lý dữ liệu khuyết dạng số:

- Xoá bỏ phần tử khuyết thiếu dữ liệu khỏi mẫu.
- Thay giá trị khuyết bằng giá trị trung bình các phần tử không khuyết của mẫu.
- Thay các giá trị khuyết bằng giá trị trung vị các phần tử không khuyết của mẫu.

Trước tiên chúng ta sẽ tiến hành trích xuất các mẫu cần kiểm tra và kiểm tra số lượng, tỷ lệ khuyết thiếu của các mẫu bằng lệnh `apply()`.

```
extract_Data <- data[,c("Name",
                        "Number_of_Pixels",
                        "Core_Speed_Value",
                        "Max_Power_Value",
                        "Memory_Value",
                        "Memory_Bandwidth_Value",
                        "Memory_Speed_Value",
                        "Manufacturer",
                        "Release_Date",
                        "Release_Price")]
apply(is.na(extract_Data),MARGIN = 2, FUN = sum)
apply(is.na(extract_Data),MARGIN = 2, FUN = mean)
```

Chúng ta sẽ trích xuất các cột có tên **Name**, **Number_of_Pixels**, **Core_Speed_Value**, **Max_Power_Value**, **Memory_Value**, **Memory_Bandwidth_Value**, **Memory_Speed_Value**, **Manufacturer**, **Release_Date** và **Release_Price** bằng toán tử truy xuất và gán các giá trị đó vào một biến `extract_Data`. Chúng ta sẽ tiến hành kiểm tra trên biến `extract_Data`.

Hàm `apply()` có nhiều tác dụng hữu ích, một trong số đó là xử lý các dữ liệu thiếu (NA). Hàm `apply()` sẽ có các tham số như `FUN` đại diện cho các hàm được áp dụng lên dữ liệu, `MARGIN` là tham số đại diện cho số chiều mà hàm `FUN` được áp dụng. Để đếm số lượng các giá trị NA trong từng cột của `extract_Data`, ta sẽ để tham số `MARGIN = 2` để áp dụng hàm `FUN` lên từng cột, `FUN = sum()` để đếm các giá trị NA. Chúng ta sẽ dùng hàm `is.na()` để xác định các giá trị NA có trong `extract_Data`. Tương tự, để kiểm tra tỷ lệ phần trăm NA trong từng cột của `extract_Data`, ta sẽ điều chỉnh hàm `FUN = mean()` để trả về tỷ lệ phần trăm giá trị NA.



```
> # Đếm số phần tử khuyết thiếu của các mẫu.  
> apply(is.na(extract_Data),MARGIN = 2, FUN = sum)  
      Name      Number_of_Pixels      Core_Speed_Value      Max_Power_Value      Memory_Value  
Memory_Bandwidth_Value      Memory_Speed_Value      Manufacturer      Release_Date      Release_Price  
      0              642              936              623              420  
      121              103              0              30              2850
```

Hình 14: Số lượng giá trị khuyết thiếu (NA) của những cột có trong mẫu thử.

```
> apply(is.na(extract_Data),MARGIN = 2, FUN = mean)  
      Name      Number_of_Pixels      Core_Speed_Value      Max_Power_Value      Memory_Value  
Memory_Bandwidth_Value      Memory_Speed_Value      Manufacturer      Release_Date      Release_Price  
0.000000000 0.188490898 0.274809160 0.183499796 0.12331803  
0.035525543 0.030827931 0.000000000 0.008807986 0.836758661
```

Hình 15: Tỷ lệ phần trăm giá trị khuyết thiếu (NA) của những cột có trong mẫu thử.

```
extract_Data <- data[,!(names(data) %in% "Release_Price")]  
extract_Data <- na.omit(extract_Data)  
#  
data <- data[rownames(extract_Data),]  
#  
apply(is.na(data),MARGIN = 2, FUN = mean)
```

Từ dữ liệu thu thập được, ta có thể thấy rằng tỷ lệ khuyết thiếu thông tin của dữ liệu tương đối cao. Ta có thể xử lý bằng cách xóa bỏ đi các quan sát bị khuyết thiếu dữ liệu bằng hàm `na.omit()`. Hàm `na.omit()` có tác dụng xóa bỏ các dòng (hoặc phần tử) có chứa giá trị NA từ một đối tượng bất kỳ. Tuy nhiên, thuộc tính **Release_Price** có lượng dữ liệu khuyết quá lớn, do đó trong quá trình xóa bỏ khuyết thiếu, chúng ta sẽ tạm thời bỏ qua thuộc tính này.

Chúng ta sẽ trích xuất vào tất cả phần tử có trong `data`, ngoại trừ **Release_Price**, lưu kết quả thu được vào biến `extract_Data`. Thực hiện hàm `na.omit()` đối với `extract_Data` để loại bỏ tất cả các dòng có chứa giá trị NA trong `extract_Data`. Sau khi đã xóa tất cả những dòng có chứa giá trị NA ngoại trừ cột **Release_Price**, sử dụng lệnh `rownames()` để cập nhật lại bảng dữ liệu `data` ban đầu bằng cách chỉ lấy những dòng tương ứng với dòng trong `extract_Data`. Điều này đảm bảo rằng `data` và `extract_Data` có cùng các dòng (các quan sát). Chúng ta sẽ dùng hàm `apply()` để kiểm tra tỷ lệ giá trị khuyết thiếu có trong từng cột.

```
> apply(is.na(data),MARGIN = 2, FUN = mean)  
      Name      Number_of_Pixels      Core_Speed_Value      Core_Speed_Unit      Max_Power_Value  
0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000  
      Max_Power_Unit      Memory_Value      Memory_Unit      Memory_Bandwidth_Value      Memory_Bandwidth_Unit  
0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000  
      Memory_Speed_Value      Memory_Speed_Unit      Manufacturer      Release_Date      Release_Price  
0.00000000 0.00000000 0.00000000 0.00000000 0.7675483
```

Hình 16: Tỷ lệ giá trị khuyết thiếu (NA) của những cột có trong mẫu thử sau xử lý.

	Name	Number_of_Pixels	Core_Speed_Value	Core_Speed_Unit	Max_Power_Value	Max_Power_Unit	Memory_Value	Memory_Unit	Memory_Bandwidth_Value
7	Radeon	2073600	870	MHz	150	Watts	2048	MB	134.4
12	Tesla	2073600	650	MHz	250	Watts	6144	MB	177.6
14	Tesla	4096000	706	MHz	245	Watts	12288	MB	208.4
16	Radeon	1682000	1050	MHz	150	Watts	3072	MB	57.6
47	GeForce	3600000	857	MHz	250	Watts	6144	MB	208.4
48	GeForce	9216000	705	MHz	375	Watts	12288	MB	672.0
50	GeForce	8294400	705	MHz	375	Watts	12288	MB	672.0
52	GeForce	8294400	1140	MHz	250	Watts	12288	MB	336.6
55	GeForce	6220800	1000	MHz	450	Watts	24576	MB	473.2
56	GeForce	8294400	1000	MHz	250	Watts	12288	MB	336.6
57	GeForce	8294400	1127	MHz	250	Watts	12288	MB	336.6
58	GeForce	3600000	838	MHz	285	Watts	6144	MB	208.4
59	GeForce	9216000	857	MHz	500	Watts	12288	MB	576.8
60	GeForce	8294400	1000	MHz	450	Watts	24576	MB	473.2

Showing 1 to 14 of 1346 entries. 15 total columns

Hình 17: Bảng dữ liệu sau quá trình xử lý.



4 Thống kê mô tả

4.1 Các đặc trưng của mẫu

Dùng các hàm `apply()`, `mean()`, `median()`, `sd()`, `min()`, `max()`, ... để thống kê các đặc trưng cơ bản của mẫu định lượng

```
data1 <- data[,c(
  "Number_of_Pixels",
  "Core_Speed_Value",
  "Max_Power_Value",
  "Memory_Value",
  "Memory_Bandwidth_Value",
  "Memory_Speed_Value",
  "Release_Date")]
aver <- apply(data1,2,mean)
s <- apply(data1,2,var)
Q1 <-apply(data1,2,function(x) quantile(x , probs = 0.25 , na.rm = TRUE))
Q2 <-apply(data1,2,function(x) quantile(x , probs = 0.5 , na.rm = TRUE))
Q3 <-apply(data1,2,function(x) quantile(x , probs = 0.75 , na.rm = TRUE))
min <-apply(data1,2,min)
max <-apply(data1,2,max)
round(data.frame(aver,s,Q1,Q2,Q3,min,max),2)
```

Kết quả khi chạy chương trình trên

	mean	s2	Q1	Q2
Number_of_Pixels	2313406.78	2.580256e+12	1440000.00	2073600.00
Core_Speed_Value	980.88	6.413929e+04	800.00	1000.00
Max_Power_Value	144.88	1.086828e+04	65.00	130.00
Memory_Value	3308.73	7.700892e+06	1280.00	2048.00
Memory_Bandwidth_Value	166.13	1.922599e+04	72.00	128.30
Memory_Speed_Value	1307.28	1.466370e+05	1000.00	1350.00
Release_Date	2013.68	4.690000e+00	2012.25	2013.83

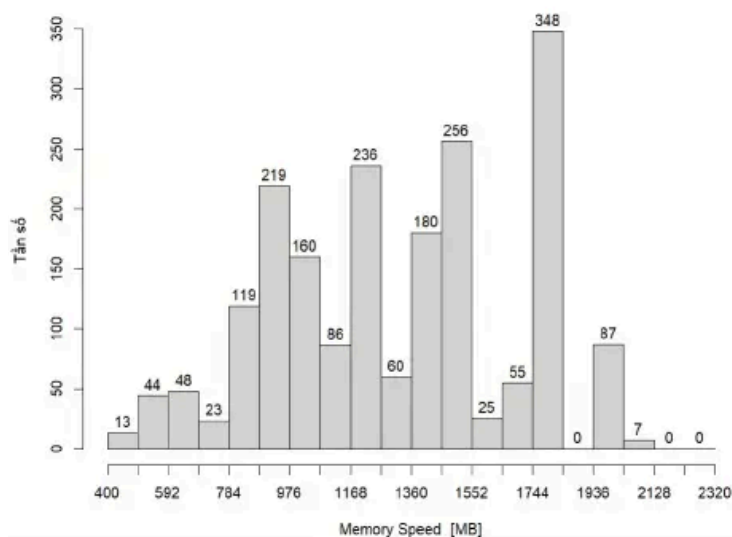
	Q3	min	max
Number_of_Pixels	2073600.00	480000.00	9216000.00
Core_Speed_Value	1105.00	200.00	1784.00
Max_Power_Value	190.00	1.00	780.00
Memory_Value	4096.00	128.00	24576.00
Memory_Bandwidth_Value	224.40	4.00	1280.00
Memory_Speed_Value	1650.00	400.00	2127.00
Release_Date	2015.42	2006.92	2017.33

Bảng 4.1: Các giá trị đặc trưng của các mẫu định lượng

4.2 Biểu đồ và đồ thị

4.2.1 Biểu đồ tần số

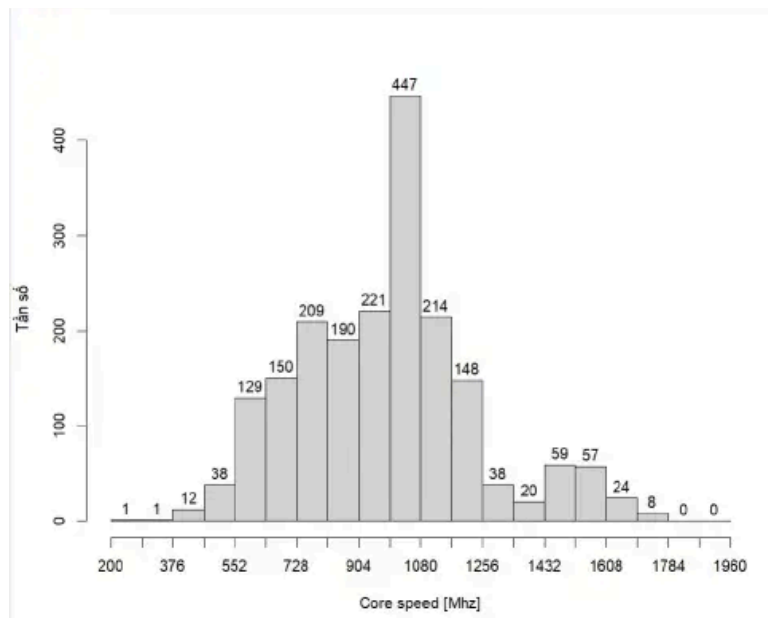
```
plotHist <- function(sample, name ) {
  x_axis = seq(floor(min(sample)),max(sample)*1.1,
    by = floor((max(sample)*1.1-min(sample))/20))
  hist(sample,main="",xlab = name,xaxt= "n",ylab="Tần số", label= TRUE,breaks
    = x_axis)
  axis(1,at = x_axis)}
plotHist (data1$Core_Speed_Value, name = "Core speed [Mhz]")
plotHist (data1$Max_Power_Value, name = "Max Power [Watts]")
plotHist (data1$Memory_Bandwidth_Value, name = "Memory Bandwidth [GB/sec]")
plotHist (data1$Memory_Speed_Value, name = "Memory Speed [MB] ")
```



Hình 18: Biểu đồ phân phối tần số của **Memory_speed**

Nhận xét:

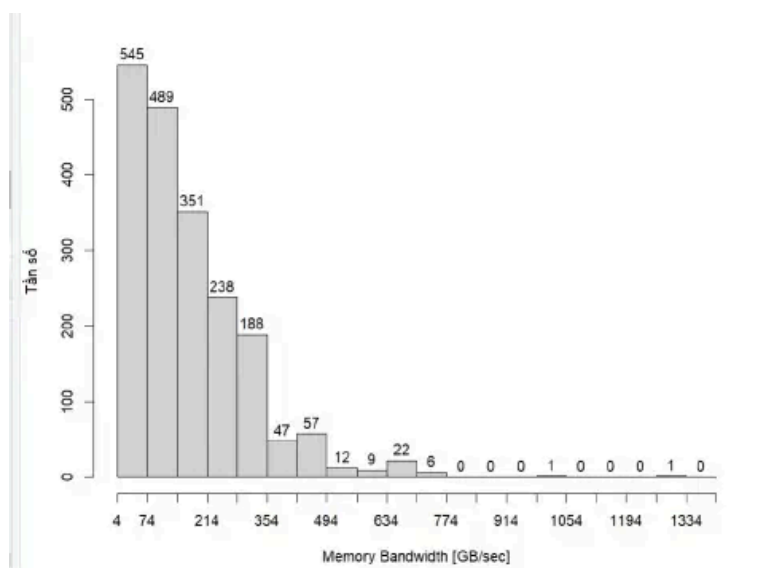
- **Memory_speed:** Phân phối tương đối chuẩn, tập trung chủ yếu trong khoảng 784 đến 1744 MHz. Các giá trị thấp hơn có thể thuộc về GPU thế hệ cũ hoặc dòng sản phẩm phổ thông, trong khi các giá trị cao hơn thường thấy ở GPU chuyên dụng cho đồ họa tiên tiến, tính toán khoa học và AI, đòi hỏi hiệu suất cực cao.



Hình 19: Biểu đồ phân phối tần số của **Core_speed**

Nhận xét:

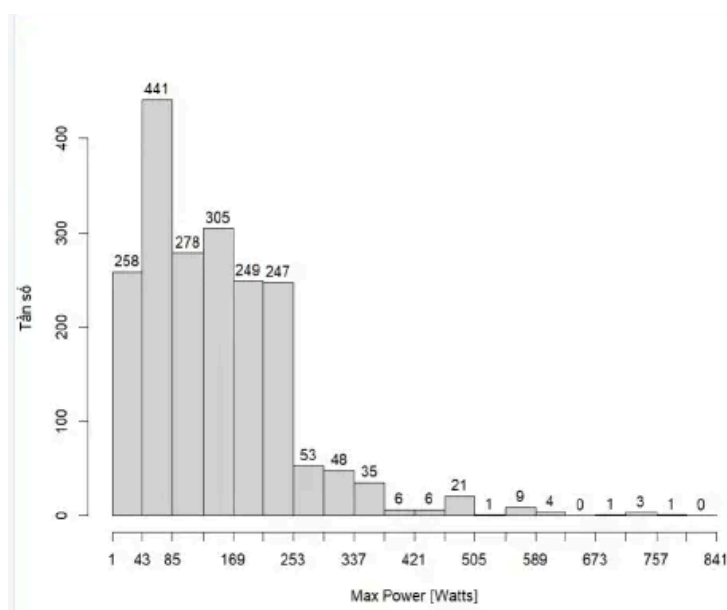
- **Core_speed:** Tương tự Memory_speed, phân phối cũng tương đối chuẩn, tập trung chủ yếu từ 552 đến 1256 MHz. Các giá trị thấp hơn có thể là GPU đời cũ hoặc dòng phổ thông, trong khi các giá trị cao hơn thường dành cho GPU chuyên xử lý tác vụ phức tạp như đồ họa chuyên nghiệp, render và mô phỏng.



Hình 20: Biểu đồ phân phối tần số của **Memory_bandwidth**

Nhận xét:

- **Memory_bandwidth:** Phân phối lệch trái, phần lớn giá trị dưới 300 GB/s. Điều này phản ánh nhu cầu thực tế của người dùng, nơi đa số sử dụng GPU cho các tác vụ như chơi game, xem phim, không đòi hỏi băng thông bộ nhớ quá cao. Các giá trị cao hơn thường dành cho GPU chuyên dụng xử lý đồ họa chuyên nghiệp, AI và tính toán khoa học.



Hình 21: Biểu đồ phân phối tần số của **Max_power**

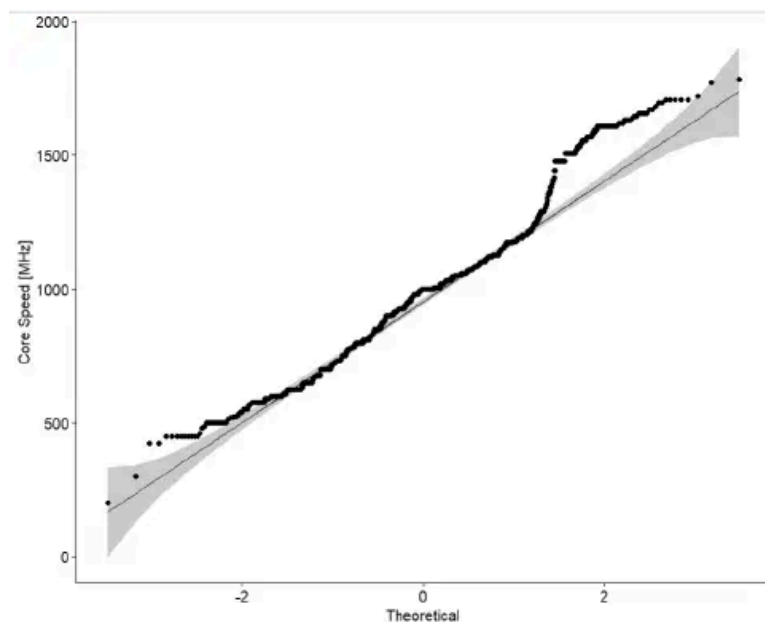
Nhận xét:

- **Max_power:** Phân phối lệch trái, phần lớn giá trị dưới 250 Watts, còn các giá trị có tần số thấp là các GPU xử lý cho tác vụ phức tạp

4.2.2 Phân phối chuẩn

Phân phối chuẩn là giả thiết quan trọng cho một số mô hình. Vì thế ta cần kiểm tra xem các yếu tố đã trích xuất có (hoặc gần với) phân phối chuẩn hay không. Để làm điều đó, ta dùng hàm `ggqqplot()` trong thư viện `ggpubr` để vẽ biểu đồ QQ-plot (biểu đồ kiểm tra mẫu có phân phối chuẩn hay không) cho các mẫu định lượng.

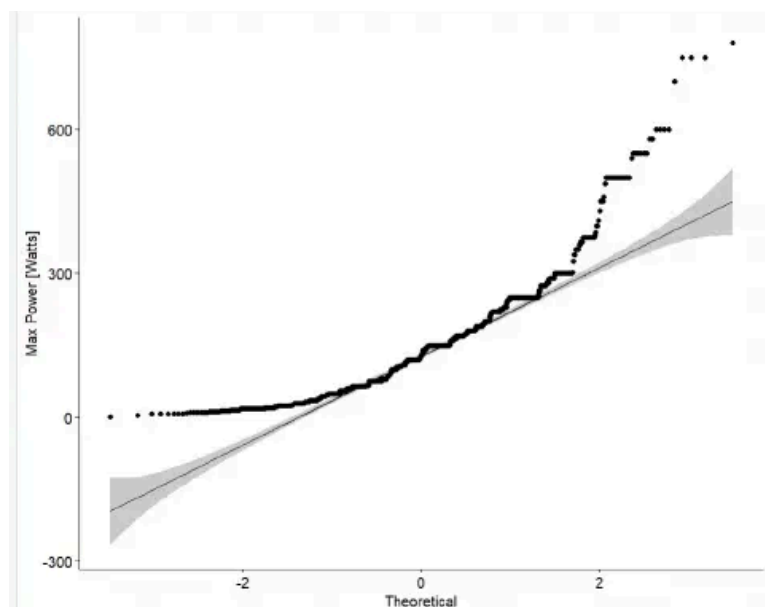
```
library(ggpubr)
ggqqplot(data1$Core_Speed_Value, ylab = "Core Speed [MHz]")
ggqqplot(data1$Max_Power_Value, ylab = "Max Power [Watts]")
ggqqplot(data1$Core_Speed_Value, ylab = "Memory Bandwidth [GB/sec]")
ggqqplot(data1$Core_Speed_Value, ylab = "Memory Speed [MB] ")
```



Hình 22: Biểu đồ QQ-plot của **Core_speed**

Nhận xét:

Phân phối **Core_speed** gần tuân theo phân phối chuẩn, tuy nhiên có một số điểm lệch nhẹ. Phần lớn các điểm dữ liệu nằm gần đường chéo, cho thấy sự tương đồng với phân phối chuẩn.

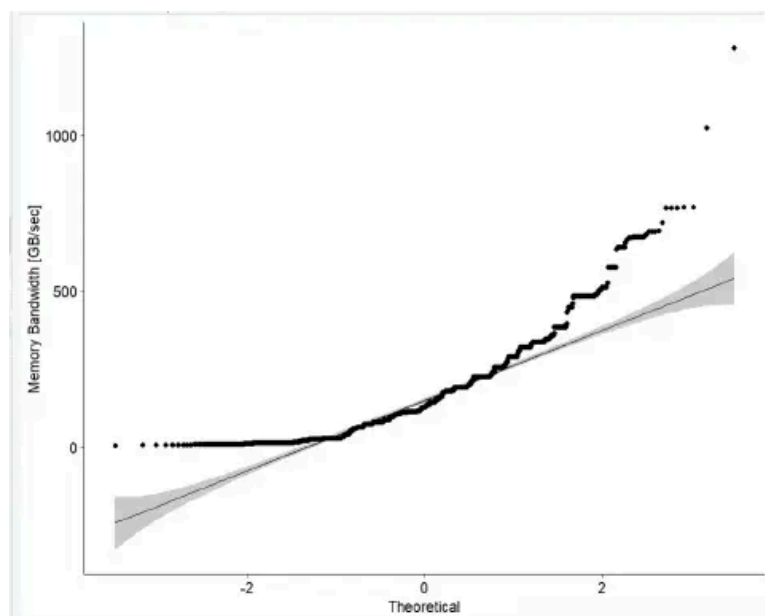


Hình 23: Biểu đồ QQ-plot của **Max_power**

Nhận xét:

Phân phối **Max_power** không tuân theo phân phối chuẩn. Q-Q plot cho thấy phần lớn các điểm

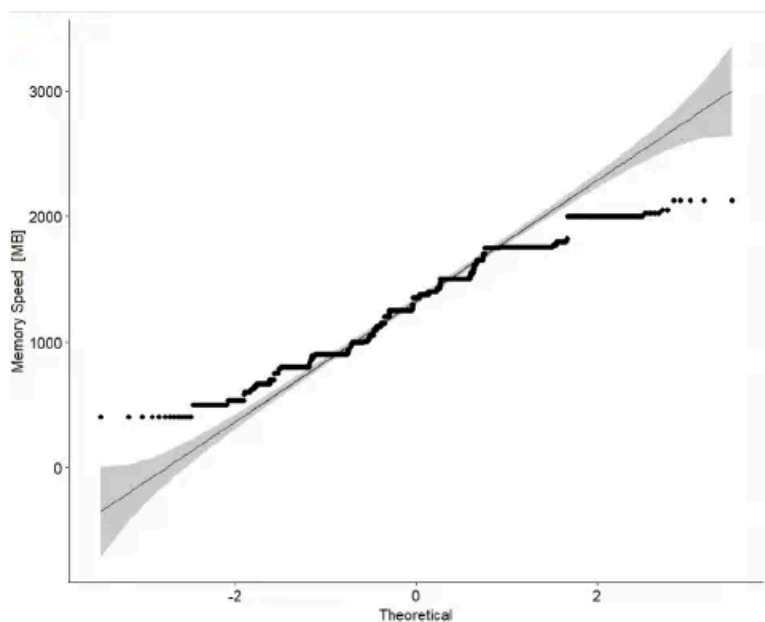
dữ liệu nằm trên đường chéo.



Hình 24: Biểu đồ QQ-plot của `Memory_bandwidth`

Nhận xét

Có khá nhiều quan sát lệch xa đường kì vọng do đó không tuân theo phân phối chuẩn.



Hình 25: Biểu đồ QQ-plot của `Memory_speed`

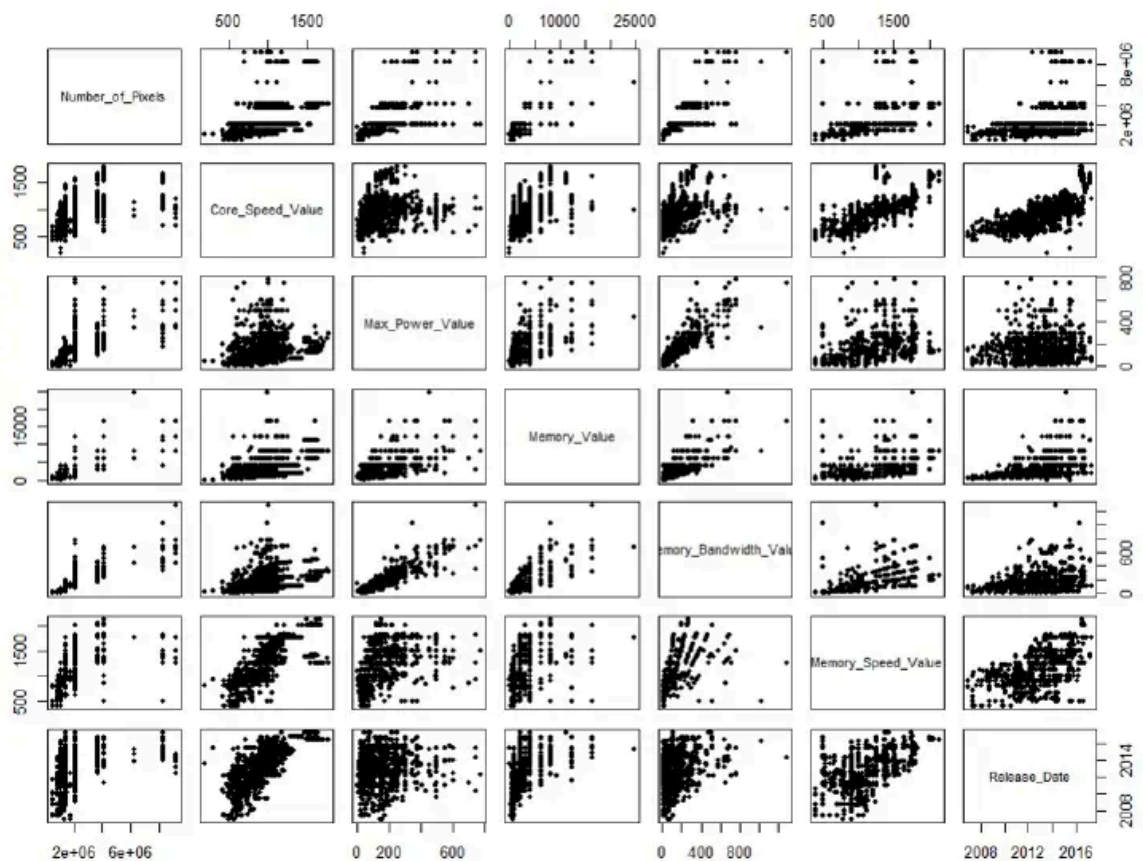
Nhận xét:

phân phối **Memory_speed** gần tuân theo phân phối chuẩn, nhưng có một số điểm lệch nhẹ ở hai đuôi, với ít giá trị thấp và nhiều giá trị cao hơn so với đường thẳng kì vọng.

4.2.3 Mối liên hệ giữa các biến

Dùng hàm **pair()** với các tham số phù hợp để vẽ ma trận các biểu đồ tán xạ thể hiện mối quan hệ giữa các mẫu định lượng với nhau.

```
pairs(data1, main="", pch=20)
```



Hình 26: Ma trận các biểu đồ tán xạ thể hiện mối quan hệ giữa các mẫu



- **Core_speed và Memory_speed:** Có mối tương quan dương rõ rệt giữa hai biến này, cho thấy các GPU có tốc độ nhân cao thường có **Memory_speed** cao.
- **Max_power và Core_speed:** Mối tương quan dương giữa hai biến này cho thấy các GPU có **Max_power** cao hơn thường có **Core_speed** cao hơn.
- **Release_date và các biến khác:** và các biến khác: Có thể thấy xu hướng chung là các GPU ra mắt sau này có xu hướng ngày càng tốt với tốc độ xử lý ngày càng cao .



5 Thống kê suy diễn

5.1 Tìm khoảng tin cậy một mẫu

5.1.1 Mục tiêu

- Nắm được các khái niệm liên quan đến lý thuyết ước lượng, bài toán ước lượng khoảng tin cậy thông số của một tổng thể.
- Biết cách giải một bài toán tìm khoảng tin cậy một mẫu.
- Tìm hiểu và hiện thực các lệnh của ngôn ngữ R để giải quyết bài toán.

5.1.2 Bài toán

Xét mẫu **Memory_speed** trong dữ liệu đã trích xuất và tiền xử lý. Tiến hành ước lượng khoảng tin cậy trung bình tổng thể của mẫu này với độ tin cậy 99%

5.1.3 Thực hiện

Nhận xét bài toán: Mẫu được trích xuất có phân phối chuẩn và không biết phương sai và có kích thước mẫu lớn. Khi mẫu lớn thì phân phối chuẩn và phân phối student xấp xỉ nhau nên ta có thể kiểm định với hàm t.test trong R.

Các bước thực hiện:

- Bước 1: Ta trích xuất mẫu **Memory_Speed_Value** từ biến data
- Bước 2: Xác định độ tin cậy
- Bước 3: Nhập các tham số vào hàm t.test và đọc kết quả

```
#trich mau Memmory_Speed tu data
sample <- data$Memory_Speed_Value
#Muc y nghĩa của kiểm định, cho truoc alpha = 1%
alpha <- 0.01
#Dung ham t.test() de kiểm định trung bình cho mot mau
t.test(sample, conf.level = 1 - alpha )
```

5.1.4 Kết quả

One Sample t-test

```
data: sample
t = 151.37, df = 1965, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
99 percent confidence interval:
 1285.010 1329.545
sample estimates:
mean of x
 1307.278
```



5.1.5 Nhận xét

Ta bỏ qua các dòng đầu tiên của kết quả (kết quả của bài toán kiểm định trung bình), ta quan tâm đến dòng **99 percent confidence interval**. Kết quả cho ta biết khoảng ước lượng hai phía cho trung bình của tổng thể của **Memory_speed** là:

(1285.010; 1329.545)

5.1.6 Kết luận

Trong bài toán này, nhóm đã tìm hiểu thành công các lệnh của R để thực hiện tính toán và tìm ra khoảng ước lượng cho trung bình tổng thể của một mẫu bất kỳ trong tập dữ liệu với mức độ tin cậy đã cho. Tuy nhiên, do tỷ lệ dữ liệu bị khuyết của mẫu khá lớn nên các quan sát bị thiếu đã bị loại bỏ. Vì vậy, kết quả mà nhóm tìm được có thể hơi khác so với kết quả chính xác.



5.2 Kiểm định hai mẫu

5.2.1 Mục tiêu

- Hiểu được khái niệm liên quan đến kiểm định giả thuyết thống kê.
- Biết cách giải một bài toán kiểm định tỉ lệ một mẫu, kiểm định tỉ lệ hai mẫu.
- Tìm hiểu và hiện thực các lệnh của ngôn ngữ R để giải quyết bài toán.

5.2.2 Bài toán

Trích xuất ra hai mẫu con từ **Memory_Speed_Value** dựa trên hai tần số cao nhất của mẫu **Manufacturer** (nhãn hàng sản xuất GPU) tương ứng. Từ hai mẫu con đã trích, hãy kiểm định với mức ý nghĩa 5%, có thể coi giá trị trung bình về **Memory_Speed** của tổng thể các GPU do hai nhãn hàng sản xuất là như nhau hay không?

5.2.3 Nhận xét bài toán

Đây là bài toán kiểm định hai mẫu độc lập (do mỗi GPU chỉ đến từ một trong hai nhãn hàng, và hai nhãn hàng này sản xuất độc lập với nhau), chưa biết phương sai tổng thể. Ta sẽ dùng R kết hợp với các lý thuyết đã học để giải quyết bài toán này.

μ_1 : Trung bình mẫu **Memory_Speed_Value** của nhãn hàng 1.

μ_2 : Trung bình mẫu **Memory_Speed_Value** của nhãn hàng 2.

- Giả thiết không $H_0: \mu_1 = \mu_2$
- Giả thiết đối $H_1: \mu_1 \neq \mu_2$

5.2.4 Kiến thức R

Trong R, có các hàm **z.test()** và **t.test()** để giúp ta kiểm định cho trung bình hai mẫu. Các hàm này cùng các tham số trong từng hàm được xây dựng dựa trên các trường hợp đã phân định trong lý thuyết. Cụ thể:

- Khi các tổng thể tuân theo phân phối chuẩn, đã biết phương sai tổng thể, ta dùng hàm **z.test()**.
- Khi các tổng thể tuân theo phân phối chuẩn, chưa biết phương sai tổng thể, ta dùng hàm **t.test()**.
- Khi các tổng thể không tuân theo phân phối chuẩn nhưng cỡ mẫu lớn ($n \geq 30$) thì theo định lý giới hạn, thống kê được sử dụng có phân phối xấp xỉ phân phối chuẩn nên ta dùng hàm **z.test()**. Tuy nhiên, khi cỡ mẫu lớn thì phân phối chuẩn và phân phối Student xấp xỉ nhau nên trong trường hợp này, ta có thể dùng phân phối Student để thay thế, tức là dùng hàm **t.test()**.

Lưu ý: Hàm **t.test()** có sẵn trong các gói cơ bản của R, còn hàm **z.test()** chỉ dùng được thông qua gói thư viện BSDA.

Để kết luận bài toán, ta dựa vào kết quả **p-value** mà hàm kiểm định trả về. Nếu:

- $p\text{-value} \leq \alpha$: ta bác bỏ giả thuyết trung bình về **Memory_Speed** của tổng thể các GPU do hai nhãn hàng sản xuất là bằng nhau, tức là thừa nhận có sự khác biệt về trung bình giá trị **Memory_Speed** của tổng thể các GPU được sản xuất từ hai nhãn hàng.
- $p\text{-value} > \alpha$: ta chưa thể bác bỏ giả thuyết trung bình về **Memory_Speed** của hai nhãn hàng là bằng nhau, tức là chưa thể kết luận về trung bình giá trị **Memory_Speed** của tổng thể các GPU được sản xuất từ hai nhãn hàng.



Nếu có sự khác biệt về trung bình giá trị **Memory_Speed** của tổng thể các GPU được sản xuất từ hai nhãn hàng và ta muốn biết cụ thể liệu trung bình giá trị **Memory_Speed** của tổng thể các GPU được sản xuất từ nhãn hàng nào lớn hơn, ta quan tâm đến giá trị **t** mà hàm kiểm định trả về. Nếu:

- $t < 0$: trung bình giá trị **Memory_Speed** của tổng thể các GPU được sản xuất từ nhãn hàng I là nhỏ hơn nhãn hàng II.
- $t > 0$: trung bình giá trị **Memory_Speed** của tổng thể các GPU được sản xuất từ nhãn hàng I là lớn hơn nhãn hàng II.

5.2.5 Tiến hành

Tiến hành trích xuất hai mẫu con từ **Memory_Speed_Value**:

- Thống kê số lượng GPU mỗi nhãn hàng sản xuất có trong bảng dữ liệu bằng hàm **table()**

```
table(data$Manufacturer)
```

Kết quả khi thực thi đoạn lệnh trên:

```
AMD      ATI      Intel Nvidia
690      65       2     1209
```

Từ thống kê, có thể thấy bảng số liệu chứa nhiều thông tin GPU đến từ **AMD** và **Nvidia** nhất.

- Do đó, ta sẽ trích xuất hai mẫu con của **Memory_Speed_Value** theo hai nhãn hàng này.

```
sample1 <- data$Memory_Speed_Value[data$Manufacturer == "AMD"]
sample2 <- data$Memory_Speed_Value[data$Manufacturer == "Nvidia"]
```

- Tiến hành kiểm định trung bình hai mẫu con vừa trích xuất:

```
alpha <- 0.05
t.test(sample1, sample2, var.equal =
  (var(sample1) == var(sample2)), conf.level = 1 - alpha,
  alternative = "two.sided")
```

5.2.6 Kết quả

```
Welch Two Sample t-test

data: sample1 and sample2
t = -8.1383, df = 1755.6, p-value = 7.507e-16
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -165.7108 -101.3500
sample estimates:
mean of x mean of y
 1204.996  1338.526
```



5.2.7 Nhận xét

Do phương sai của hai mẫu khác biệt nhau (phép so sánh `var(sample1) == var(sample2)` trả về **FALSE**) nên hàm `t.test()` sử dụng **Welch Two Sample t-test** để kiểm định cho bài toán này.

Ta quan tâm đến giá trị **p-value**. Do $p\text{-value} = 7.507e - 16$, bé hơn rất nhiều so với giá trị $\alpha = 5\%$. Do đó, ta kết luận có sự khác biệt về trung bình giá trị **Memory_Speed** của tổng thể các GPU được sản xuất bởi hai nhãn hàng **AMD** và **Nvidia**.

Cụ thể, giá trị $t < 0$, do đó, kết luận trung bình giá trị **Memory_Speed** của tổng thể các GPU được sản xuất bởi nhãn hàng **AMD** là thấp hơn nhãn hàng **Nvidia**.

Ngoài ra, hàm kiểm định còn cung cấp khoảng tin cậy hai phía cho chênh lệch giữa trung bình giá trị **Memory_Speed** của tổng thể các GPU được sản xuất bởi nhãn hàng **AMD** so với nhãn hàng **Nvidia** với độ tin cậy $1 - \alpha = 5\%$:

$(-165.7108; -101.3500)$

5.2.8 Kết luận

Trong bài toán này, nhóm đã thành công trong việc tìm hiểu các lệnh của R để thực hiện việc tính toán, kiểm định được trung bình tổng thể của hai mẫu độc lập trong tập dữ liệu với mức ý nghĩa. Tuy nhiên, do tỉ lệ khuyết dữ liệu của mẫu khá lớn, quá trình tiền xử lý dữ liệu nên nhìn chung, kết quả mà nhóm tìm được có thể sai khác so với kết quả chính xác.



5.3 Phân tích phương sai ANOVA 1 yếu tố

5.3.1 Mục tiêu

- Hiểu được các khái niệm liên quan đến phân tích phương sai ANOVA.
- Biết cách giải một bài toán phân tích phương sai một yếu tố.
- Tìm hiểu và hiện thực các lệnh của ngôn ngữ R để giải quyết bài toán.

5.3.2 Bài toán

Dùng mô hình ANOVA kiểm định trung bình của yếu tố giữa các nhóm có **Core_speed** giữa các nhóm có **Name** khác nhau với mức ý nghĩa $\alpha = 5\%$

Với H_0 : Không có sự khác biệt giữa các giá trị trung bình của **Core_speed** giữa các nhóm, tức là:

$$\mu_1 = \mu_2 = \dots = \mu_k$$

H_1 : Có ít nhất một cặp giá trị trung bình của **Core_speed** khác nhau giữa các nhóm, tức là:

$$\exists i \neq j : \mu_i \neq \mu_j$$

5.3.3 Kiểm tra giả thiết

Như đã trình bày ở phần Thống kê tả, trong các mẫu định lượng thì yếu tố **Core_speed** là một trong những mẫu có phân phối gần với dạng chuẩn, đây là giả thiết cần cho mô hình ANOVA. Tiếp theo, ta cần kiểm tra sự tương đồng về phương sai mẫu giữa các nhóm quan sát. Để làm điều đó, ta có thể dùng kiểm định Barlett hoặc kiểm định Levene.

Tiếp theo, do ta phân loại các mẫu với nhau thông qua tên của chúng, nên có thể coi các mẫu quan sát được lấy một cách độc lập.

5.3.4 Quy trình giải quyết bài toán

- Kiểm tra - xử lý mẫu phân loại

Dùng hàm `table()` để thống kê số lượng quan sát tương ứng với từng tên trong mẫu **Name**

```
#Lưu kết quả của hàm table() vào biến rồi in kết quả ra
print(name_table <- table(data$name))
```

Kết quả sau khi thực thi đoạn chương trình trên:

```
FirePro  GeForce  GeForce  Iris Mobility  Quadro  Radeon  Tesla
      3      1      1188      2      11      18      741      2
```

Ta thấy, có một vài loại GPU có số lượng quan sát khá thấp. Do đó, để mô hình ANOVA chính xác hơn, ta sẽ lọc bỏ các quan sát có số lượng không lớn hơn.

```
# Trích xuất dữ liệu với số lượng mỗi quan sát theo tên nhiều hơn 10
data_filtered <- data[data$name %in% names(name_table[name_table > 10]), ]
# Kiểm tra lại thông kê các tên
table(data_filtered$name)
```

Kết quả sau khi thực thi đoạn chương trình trên:

```
GeForce Mobility  Quadro  Radeon
1188      11      18      741
```



Sau khi thực thi đoạn chương trình trên, ta còn lại bốn nhóm GPU. Đây sẽ là bốn nhóm phân loại cho đầu vào của mô hình ANOVA.

- Kiểm tra giả thiết để áp dụng mô hình ANOVA

Như đã trình bày, ta đã có giả thiết về phân phối chuẩn cũng như cách chọn mẫu độc lập. Do đó, ta cần kiểm định thêm về sự tương đồng của phương sai các nhóm thông qua hàm `bartlett.test()`.

```
bartlett.test(Memory_Speed_Value ~ Name, data = data_filtered)
```

Kết quả sau khi thực thi:

```
Bartlett test of homogeneity of variances

data: Core_Speed_Value by Name
Bartlett's K-squared = 230.33, df = 3, p-value < 2.2e-16
```

Nhận xét: p-value của kiểm định rất nhỏ so với mức ý nghĩa $\alpha = 5\%$. Điều này có nghĩa ta bác bỏ giả thuyết về sự bằng nhau giữa phương sai các nhóm. Nói cách khác, có sự khác biệt về phương sai giữa các loại GPU với nhau. Trên lý thuyết, mẫu này không thỏa giả thiết có thể áp dụng mô hình ANOVA, tuy nhiên với giới hạn về mặt dữ liệu, ta giả sử mẫu thỏa giả thiết và tiếp tục thực hiện mô hình.

- Tiến hành phân tích phương sai

Dùng hàm `aov()` kết hợp `summary()` với những tham số phù hợp để tiến hành áp dụng mô hình phân tích phương sai một yếu tố cho mẫu **Core_Speed_Value**, phân loại theo **Name** đã lọc.

```
# Lưu kết quả của mô hình ANOVA vào biến (để sử dụng sau này)
anova_result <- aov(Core_Speed_Value ~ Name, data = data_filtered)
# Hiện thị kết quả chi tiết của mô hình ANOVA
summary(anova_result)
```

Kết quả sau khi thực thi:

```
          Df    Sum Sq Mean Sq F value Pr(>F)
Name       3   8528849  2842950   47.91 <2e-16 ***
Residuals 1954 115947244   59338
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Nhận xét: Giá trị p-value thấp hơn rất nhiều so với mức ý nghĩa $\alpha = 5\%$ cho thấy rõ ràng sự khác biệt về trung bình giữa các nhóm được quan sát là có ý nghĩa thống kê. Tóm lại, kết quả mô hình ANOVA cho bài toán này dẫn đến việc bác bỏ giả thuyết không H_0 - giả thiết cho rằng giá trị **Core_speed** trung bình của các nhóm đều bằng nhau, hay còn có nghĩa là tốc độ Core trung bình của các nhóm là khác nhau.

- Phân tích sâu ANOVA:

Do giả thuyết không H_0 đã được bác bỏ, ta có nhu cầu kiểm định giả thuyết về trung bình **Core_Speed** giữa từng nhóm. Sử dụng hàm `LSD.test()` trong thư viện `agricolae` để xem kết quả về các giả thiết trung bình tổng thể giữa các nhóm.

```
library(agricolae)
# Dùng LSD.test() cho kết quả của mô hình ANOVA để kiểm định
# trung bình giữa
print(LSD.test(anova_result, "Name", alpha = 0.05))
```

Kết quả sau khi thực thi:



```
$groups
  Core_Speed_value groups
GeForce      1029.8426    a
Radeon       918.5007    b
Quadro       710.2222    c
Mobility      613.6364    c

attr(,"class")
[1] "group"
```

Nhận xét: Có thể thấy bốn tên GPU được chia thành ba nhóm a, b và c. Trong đó, **Quadro** và **Mobility** thuộc cùng một nhóm, còn **GeForce** và **Radeon** thuộc hai nhóm riêng, độc lập. Điều này thể hiện rằng với mức ý nghĩa $\alpha = 5\%$ chỉ có trung bình **Core_speed** của tổng thể hai nhóm **Quadro** và **Mobility** bằng nhau, còn giữa các cặp nhóm còn lại đều có sự khác biệt. Để chắc chắn hơn vào kết luận trên, ta có thể dùng hàm `pairwise.t.test()` để xem ma trận các p-value của kiểm định Student theo cặp với độ lệch chuẩn gộp giữa các nhóm.

```
pairwise.t.test(data_filtered$Core_Speed_Value, data_filtered$Name, p.adjust.method
= "none")
```

Kết quả sau khi thực thi:

```
Pairwise comparisons using t tests with pooled SD

data: data_filtered$Core_Speed_Value and data_filtered$Name

      GeForce Mobility Quadro
Mobility 1.9e-08 -      -
Quadro   3.7e-08 0.30031 -
Radeon   < 2e-16 3.9e-05 0.00035

P value adjustment method: none
```

Nhận xét: Ma trận p-value giữa các nhóm từ hàm `pairwise.t.test()` cho thấy chỉ có p-value giữa **Quadro** và **Mobility** (bằng 0.30031) là lớn hơn mức ý nghĩa $\alpha = 5\%$. Điều này phù hợp với kết luận đã đưa ra bên trên.

5.3.5 Kết luận

Trong bài toán này, nhóm đã thành công trong việc nghiên cứu các lý thuyết về phân tích phương sai ANOVA, tìm hiểu các lệnh của R để thực hiện việc tính toán, áp dụng mô hình ANOVA cho một yếu tố cụ thể, đồng thời kiểm tra sâu được giả thuyết về trung bình giữa các nhóm của yếu tố, đưa ra được kết luận cuối cùng cho bài toán. Tuy nhiên, kết quả nhóm trình bày có thể chưa chính xác, có thể là do ảnh hưởng của quá trình loại bỏ các quan sát khuyết trong quá trình xử lý số liệu hay sự chênh lệch lớn giữa số lượng phần tử giữa các nhóm nên nhìn chung kết quả mà nhóm tìm được có thể có sai khác so với kết quả chính xác



5.4 Phân tích phương sai 2 yếu tố (two-way ANOVA)

5.4.1 Bài toán

Trong phần nghiên cứu này, chúng tôi sẽ sử dụng ANOVA 2 yếu tố không lặp để phân tích xem liệu **Nhà sản xuất (Manufacturer)** và **Tốc độ nhân (Core_Speed_Value)** có ảnh hưởng đến **Giá phát hành (Release_Price)** hay không?

Giả thiết thống kê được thực hiện với mức ý nghĩa $\alpha = 0.05$.

- H_0 :

1. H_{0A} : Không có sự khác biệt giữa các giá trị trung bình của **Release_Price** giữa các nhóm **Manufacturer**, tức là:

$$\mu_{A1} = \mu_{A2} = \dots = \mu_{Ak}$$

2. H_{0B} : Không có sự khác biệt giữa các giá trị trung bình của **Release_Price** giữa các nhóm **Core_Speed_Value**, tức là:

$$\mu_{B1} = \mu_{B2} = \dots = \mu_{Bl}$$

3. H_{0AB} : Không có sự tương tác giữa **Manufacturer** và **Core_Speed_Value** đối với **Release_Price**, tức là:

Không có tác động tương tác giữa Manufacturer và Core_Speed_Value

- H_1 :

1. H_{1A} : Có ít nhất một cặp giá trị trung bình của **Release_Price** khác nhau giữa các nhóm **Manufacturer**, tức là:

$$\exists i \neq j : \mu_{Ai} \neq \mu_{Aj}$$

2. H_{1B} : Có ít nhất một cặp giá trị trung bình của **Release_Price** khác nhau giữa các nhóm **Core_Speed_Value**, tức là:

$$\exists i \neq j : \mu_{Bi} \neq \mu_{Bj}$$

3. H_{1AB} : Có sự tương tác giữa **Manufacturer** và **Core_Speed_Value** đối với **Release_Price**, tức là:

$\exists (i, j) : \text{Tác động tương tác giữa Manufacturer và Core_Speed_Value tồn tại}$

5.4.2 Kiểm tra điều kiện cho dữ liệu

Trước khi tiến hành phân tích ANOVA 2 chiều, chúng tôi cần kiểm tra các điều kiện cần thiết cho dữ liệu thì **Biến phụ thuộc: Release_Price** là một biến không có phân phối chuẩn, tồn tại nhiều khiếm khuyết dữ liệu và có các điểm dị biệt.

$\Rightarrow$ Như vậy các điều kiện giả thiết về dữ liệu không được thỏa mãn, do đó kết quả phân tích ANOVA có thể không chính xác và chỉ mang tính chất tham khảo.

5.4.3 Tiến hành phân tích

Chúng tôi tiến hành phân tích ANOVA 2 chiều theo các bước sau:



1. Chuyển đổi các biến **Manufacturer** và **Core_Speed_Value** thành các yếu tố (factors) để đảm bảo tính phù hợp với mô hình ANOVA.

```
data$Manufacturer <- as.factor(data$Manufacturer)
data$Core_Speed_Value <- as.factor(data$Core_Speed_Value)
```

2. Sử dụng hàm `aov()` để thực hiện phân tích ANOVA 2 chiều, kiểm tra sự ảnh hưởng của **Manufacturer** và **Core_Speed_Value** lên **Release_Price**.

```
anova_result <- aov(Release_Price ~ Manufacturer
*Core_Speed_Value, data = data)
```

Hiển thị kết quả của phân tích ANOVA: Sử dụng `summary()` để hiển thị các giá trị F-statistic và p-value cho từng yếu tố và sự tương tác giữa chúng

```
summary(anova_result)
```



Sau khi thực thi lệnh và kết quả nhận được:

```

              Df      Sum Sq Mean Sq F value Pr(>F)
Manufacturer    2    8119167  4059583 102.222 <2e-16 ***
Core_speed_value 132  241797017 1831796  46.125 <2e-16 ***
Manufacturer:Core_Speed_Value 10   1898816   189882   4.781  2e-06 ***
Residuals      332   13184834    39713
---
signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
2929 observations deleted due to missingness
```

Hình 27: Phân tích ANOVA 2-WAY

Kết quả và nhận xét:

Kết quả phân tích ANOVA hai yếu tố được trình bày trong Bảng 27. Kết quả cho thấy rằng cả ba yếu tố **Manufacturer**, **Core_Speed_Value**, và sự tương tác giữa chúng đều có ảnh hưởng đáng kể đến **Release_Price** với mức ý nghĩa $\alpha = 0.05$. Dưới đây là các nhận xét chi tiết:

- **Manufacturer:**
 - Với **Df = 2**, có 2 mức độ khác nhau trong yếu tố **Manufacturer**.
 - Giá trị **F = 102.222** và **Pr(>F) < 2e-16***** cho thấy rằng sự khác biệt giữa các mức độ của **Manufacturer** có ảnh hưởng đáng kể đến **Release_Price**. Do đó, ta bác bỏ giả thuyết không H_0 cho yếu tố này.
- **Core_Speed_Value:**
 - Với **Df = 132**, có 132 mức độ khác nhau trong yếu tố **Core_Speed_Value**.
 - Giá trị **F = 46.125** và **Pr(>F) < 2e-16***** cho thấy rằng sự khác biệt giữa các mức độ của **Core_Speed_Value** có ảnh hưởng đáng kể đến **Release_Price**. Do đó, ta bác bỏ giả thuyết không H_0 cho yếu tố này.
- **Tương tác giữa Manufacturer và Core_Speed_Value:**
 - Với **Df = 10**, có 10 mức độ tương tác giữa hai yếu tố **Manufacturer** và **Core_Speed_Value**.
 - Giá trị **F = 4.781** và **Pr(>F) = 2e-06***** cho thấy rằng có sự tương tác có ý nghĩa thống kê giữa hai yếu tố này và ảnh hưởng đến **Release_Price**. Do đó, ta bác bỏ giả thuyết không H_0 cho sự tương tác này.
- **Residuals:**
 - Số dư tự do còn lại là **Df = 332**.
 - Tổng bình phương của sai số là **Sum Sq = 13184834**.
- **Các quan sát bị loại bỏ:**
 - Có 2929 quan sát bị loại bỏ do thiếu dữ liệu, điều này có thể ảnh hưởng đến kết quả phân tích và cần được lưu ý trong quá trình đánh giá.

Từ những kết quả trên, chúng tôi kết luận rằng cả yếu tố **Manufacturer**, **Core_Speed_Value**, và sự tương tác giữa chúng đều có ảnh hưởng quan trọng và có ý nghĩa thống kê đối với **Release_Price**. Điều này cho thấy rằng các yếu tố này cần được cân nhắc kỹ lưỡng khi định giá sản phẩm.

3. Kiểm tra leverage để lọc bỏ các điểm có leverage cao, nhằm tránh các lỗi liên quan trong quá trình vẽ biểu đồ chẩn đoán. Sau đó, thực hiện lại phân tích ANOVA với dữ liệu đã được lọc.



```
leverage <- hatvalues(anova_result)
high_leverage_points <- which(leverage > (2 * mean(leverage)))
filtered_data <- data[-high_leverage_points, ]
anova_result_filtered <- aov(Release_Price ~ Manufacturer *
  Core_Speed_Value, data = filtered_data)
summary(anova_result_filtered)
```

Kết quả nhận được:

```

              Df    Sum Sq Mean Sq F value    Pr(>F)
Manufacturer    2    7980053  3990027  101.269 < 2e-16 ***
Core_speed_value 132  241663552  1830784   46.466 < 2e-16 ***
Manufacturer:Core_speed_value 10   1898816   189882    4.819 1.75e-06 ***
Residuals      331  13041456    39400
---
signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
2873 observations deleted due to missingness
```

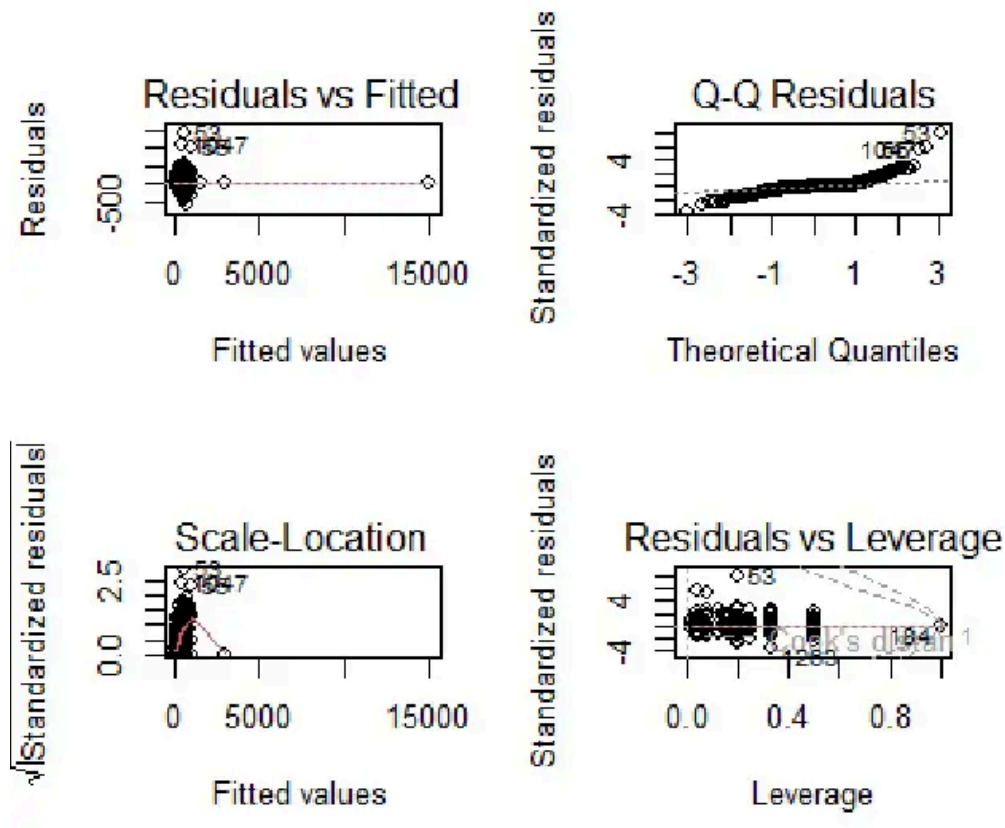
Hình 28: Kết quả sau khi lọc leverage cao

4. So sánh kết quả sau khi lọc các điểm có leverage cao:

- **Manufacturer:**
 - Sau khi lọc, giá trị F giảm nhẹ từ 102.222 xuống 101.269, nhưng p-value vẫn nhỏ hơn $2e-16$, cho thấy **Manufacturer** vẫn có ảnh hưởng đáng kể đến **Release_Price**.
- **Core_Speed_Value:**
 - Giá trị F của **Core_Speed_Value** tăng nhẹ từ 46.125 lên 46.466, và p-value vẫn nhỏ hơn $2e-16$, khẳng định rằng **Core_Speed_Value** có ảnh hưởng quan trọng đến **Release_Price**.
- **Tương tác giữa Manufacturer và Core_Speed_Value:**
 - Giá trị F của sự tương tác giữa **Manufacturer** và **Core_Speed_Value** tăng từ 4.781 lên 4.819, và p-value giảm từ $2e-06$ xuống $1.75e-06$, cho thấy rằng sự tương tác giữa hai yếu tố này vẫn có ý nghĩa thống kê sau khi lọc các điểm có leverage cao.
- **Residuals:**
 - Số dư tự do còn lại sau khi lọc là **Df = 331**, với tổng bình phương của sai số là **Sum Sq = 13041456**.

Kết quả này cho thấy rằng việc loại bỏ các điểm có leverage cao không ảnh hưởng nhiều đến kết quả phân tích tổng thể, khẳng định sự ổn định của mô hình ANOVA trong nghiên cứu này.

Mặt khác, nhìn vào biểu đồ chẩn đoán:



Hình 29: Biểu đồ chẩn đoán ANOVA

Các biểu đồ chẩn đoán cho thấy phần dư không hoàn toàn tuân theo phân phối chuẩn và có thể có vấn đề về phương sai không đồng nhất. Tuy nhiên, không có điểm nào có leverage quá cao gây ảnh hưởng mạnh đến mô hình. Điều này gợi ý rằng mô hình ANOVA có một số hạn chế nhưng vẫn đáng tin cậy.

5.4.4 Kết luận

Phân tích ANOVA 2 chiều cho thấy **Manufacturer**, **Core_Speed_Value**, và sự tương tác giữa chúng đều có ảnh hưởng đáng kể đến **Release_Price**. Việc loại các điểm có leverage cao không làm thay đổi đáng kể kết quả, khẳng định tính ổn định của mô hình. Mặc dù có một số hạn chế về phân phối chuẩn và phương sai không đồng nhất, kết quả này vẫn cung cấp cơ sở tin cậy để đánh giá và điều chỉnh chiến lược giá sản phẩm.

5.5 Hồi quy tuyến tính đa biến

Trước tiên, để xây dựng một mô hình hồi quy tuyến tính đa biến để phân tích các yếu tố ảnh hưởng đến giá cả phát hành của sản phẩm. Để phân thực hiện phân tích, ta có các biến sau:

- Biến phụ thuộc: **Release_Price**



- Các biến độc lập: `Number_of_Pixels`, `Core_Speed_Value`, `Max_Power_Value`, `Memory_Value`, `Memory_Bandwidth_Value`, `Memory_Speed_Value`

5.5.1 Tiền xử lý số liệu

Lấy ra dữ liệu từ các biến đã nêu trên và lưu nó vào một tập dữ liệu tạm thời `temp_Data`:

```
temp_Data <- data[,c("Number_of_Pixels",  
                    "Core_Speed_Value",  
                    "Max_Power_Value",  
                    "Memory_Value",  
                    "Memory_Bandwidth_Value",  
                    "Memory_Speed_Value",  
                    "Release_Price")]
```

Loại bỏ các hàng chứa những quan sát bị khuyết trong biến `Release_Price`:

```
temp_Data <- temp_Data[!is.na(temp_Data$Release_Price), ]
```

5.5.2 Xây dựng mô hình hồi quy

Trong **R** hỗ trợ xây dựng mô hình hồi quy thông qua hàm `lm()`

Giả thuyết H_0 : Biến độc lập không có ý nghĩa thống kê

```
mo.hinh.1<-lm(Release_Price~Number_of_Pixels+Core_Speed_Value+Max_Power_Value  
+Memory_Value+Memory_Bandwidth_Value+Memory_Speed_Value, data = temp_Data)  
summary(mo.hinh.1)
```

```
Call:  
lm(formula = Release_Price ~ Number_of_Pixels + Core_Speed_Value +  
    Max_Power_Value + Memory_Value + Memory_Bandwidth_Value +  
    Memory_Speed_Value, data = temp_Data)  
  
Residuals:  
    Min       1Q   Median       3Q      Max   
-1697.0  -157.1    17.2   144.9 11318.4  
  
Coefficients:  
              Estimate Std. Error t value Pr(>|t|)      
(Intercept)  8.778e+01  1.824e+02   0.481  0.63053      
Number_of_Pixels -4.677e-05  3.958e-05  -1.182  0.23791      
Core_Speed_Value  2.936e-02  1.627e-01   0.180  0.85688      
Max_Power_Value  5.433e+00  6.185e-01   8.784 < 2e-16 ***  
Memory_Value    1.146e-01  1.856e-02   6.175 1.46e-09 ***  
Memory_Bandwidth_Value -2.718e+00  6.054e-01  -4.489 9.06e-06 ***  
Memory_Speed_Value -2.838e-01  1.027e-01  -2.763 0.00595 **  
---  
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
  
Residual standard error: 621.3 on 458 degrees of freedom  
Multiple R-squared:  0.3232,    Adjusted R-squared:  0.3144   
F-statistic: 36.46 on 6 and 458 DF,  p-value: < 2.2e-16
```

So sánh các giá trị `p_value` của các biến độc lập với mức ý nghĩa 5%. Với các giá trị `p_value` của các biến lần lượt là:

- `Max_Power_Value`: $< 2e - 16$



- Memory_Value: $1.46e - 9$
- Memory_Bandwidth_Value: $9.06e - 6$
- Memory_Speed_Value: 0.00595

Các biến trên có giá trị p_value rất thấp, nên ta bác bỏ H_0 , thừa nhận các biến đó có ý nghĩa cho mô hình.

Giữ lại các biến Max_Power_Value, Memory_Value, Memory_Bandwidth_Value, Memory_Speed_Value, xây dựng lại mô hình hồi quy tuyến tính một lần nữa.

```
mo.hinh.2<-lm(Release_Price~Max_Power_Value+Memory_Value  
+Memory_Bandwidth_Value+Memory_Speed_Value, data = temp_Data)  
summary(mo.hinh.2)
```

```
Call:
lm(formula = Release_Price ~ Max_Power_Value + Memory_Value +
    Memory_Bandwidth_Value + Memory_Speed_Value, data = temp_Data)

Residuals:
    Min       1Q   Median       3Q      Max 
-1696.7  -171.8    23.2   128.6 11420.4 

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)   67.23874   148.92983    0.451  0.65186
Max_Power_Value    5.50416    0.57756    9.530 < 2e-16 ***
Memory_Value      0.10556    0.01614    6.541 1.63e-10 ***
Memory_Bandwidth_Value -3.08829    0.51468   -6.000 3.99e-09 ***
Memory_Speed_Value -0.26513    0.09093   -2.916 0.00372 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 620.9 on 460 degrees of freedom
Multiple R-squared:  0.3212,    Adjusted R-squared:  0.3152 
F-statistic: 54.4 on 4 and 460 DF,  p-value: < 2.2e-16
```

Tuy nhiên, giá trị p_value của hệ số chặn trong mô hình này là rất lớn. Nên hệ số chặn không có ý nghĩa trong mô hình này. Vì vậy, để tìm ra được mô hình tốt nhất, nhóm nghiên cứu xem xét xây dựng lại mô hình một lần nữa nhưng sẽ loại đi biến độc lập có p_value cao nhất là biến Memory_Speed_Value, sau đó sẽ so sánh hiệu quả của hai mô hình.

Thực hiện xây dựng lại mô hình với các biến độc lập: Max_Power_Value, Memory_Value, Memory_Bandwidth_Value

```
mo.hinh.3<-lm(Release_Price~Max_Power_Value+Memory_Value  
+Memory_Bandwidth_Value, data = temp_Data)  
summary(mo.hinh.3)
```



```
Call:
lm(formula = Release_Price ~ Max_Power_Value + Memory_Value +
    Memory_Bandwidth_Value, data = temp_Data)

Residuals:
    Min       1Q   Median       3Q      Max
-1619.6  -130.1    -6.6   132.5 11584.9

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)   -330.97093    59.88174  -5.527 5.46e-08 ***
Max_Power_Value    5.93254     0.56308  10.536 < 2e-16 ***
Memory_Value      0.08664     0.01490   5.816 1.13e-08 ***
Memory_Bandwidth_Value -3.05227    0.51870  -5.884 7.69e-09 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 625.9 on 461 degrees of freedom
Multiple R-squared:  0.3086,    Adjusted R-squared:  0.3041
F-statistic: 68.59 on 3 and 461 DF,  p-value: < 2.2e-16
```

Ta so sánh mô hình 2 và mô hình 3:

- Giả thuyết H0: Hai mô hình có hiệu quả giống nhau
- Giả thuyết H1: Hai mô hình có hiệu quả khác nhau

```
anova(mo.hinh.2, mo.hinh.3)
```

```
Analysis of Variance Table

Model 1: Release_Price ~ Max_Power_Value + Memory_Value + Memory_Bandwidth_Value +
    Memory_Speed_Value
Model 2: Release_Price ~ Max_Power_Value + Memory_Value + Memory_Bandwidth_Value
  Res.Df    RSS Df Sum of Sq    F    Pr(>F)
1     460 177344578
2     461 180622250 -1    -3277673  8.5017 0.003721 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Nhận xét: Giá trị $p_value = 0.0037$ thấp hơn mức ý nghĩa 5% nên ta bác bỏ H_0 . Do đó, có thể kết luận rằng hai mô hình có hiệu quả khác nhau. Mặc khác quan sát giá trị p_value của hệ số chặn Intercept và các biến độc lập trong mô hình 3 đều thấp hơn mức ý nghĩa 5% nên chứng tỏ rằng mô hình hồi quy thực sự hiệu quả.

Từ cột Estimate trong kết quả của mô hình 3, xác định được đường thẳng hồi quy ước lượng được cho bởi phương trình:

$$\text{Release_Price} = -330.97093 + 5.93254 \text{ Max_Power_Value} + 0.08664 \text{ Memory_Value} - 3.05227 \text{ Memory_Bandwidth_Value}$$

5.5.3 Ước lượng khoảng tin cậy của các hệ số

Trong R hỗ trợ hàm `confint()` để tìm khoảng ước lượng cho các hệ số của mô hình hồi quy tuyến tính với độ tin cậy 95%

```
confint(mo.hinh.3)
```



	2.5 %	97.5 %
(Intercept)	-448.64593710	-213.2959245
Max_Power_Value	4.82601322	7.0390681
Memory_Value	0.05736377	0.1159078
Memory_Bandwidth_Value	-4.07156781	-2.0329630

Từ kết quả trên, ta xác định được khoảng tin cậy của hệ số chặn β_0 cũng như hệ số của các biến độc lập trong mô hình với độ tin cậy 95%.

- Hệ số chặn β_0 : (-448.6459; -213.2959)
- Max_Power_Value: (4.8260; 7.0391)
- Memory_Value: (0.0574; 0.1159)
- Memory_Bandwidth_Value: (-4.0716; -2.0330)

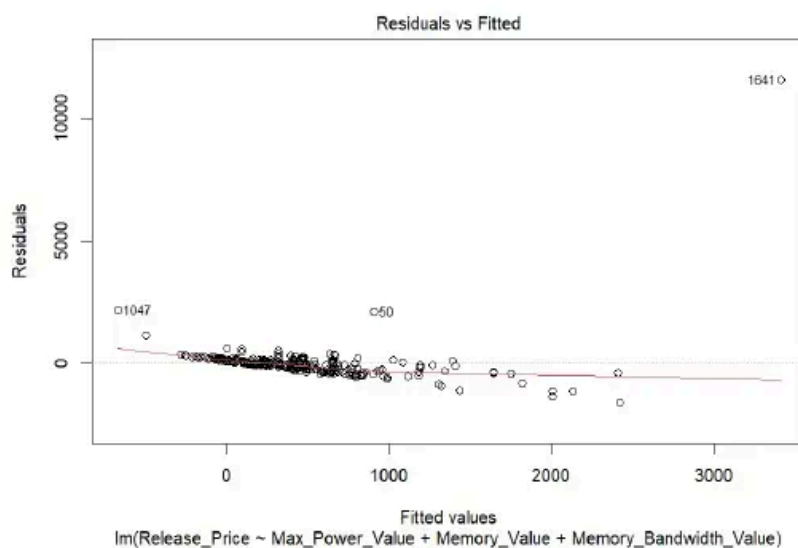
5.5.4 Kiểm tra các giả định của mô hình

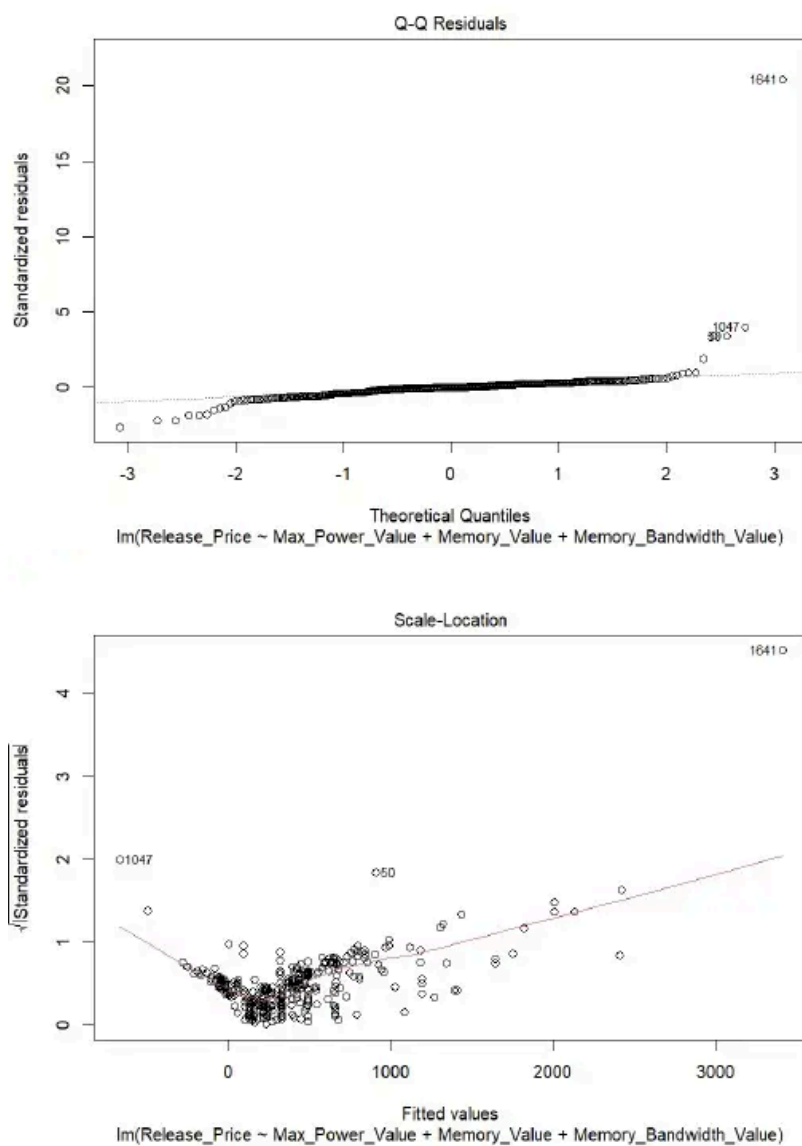
Các giả định:

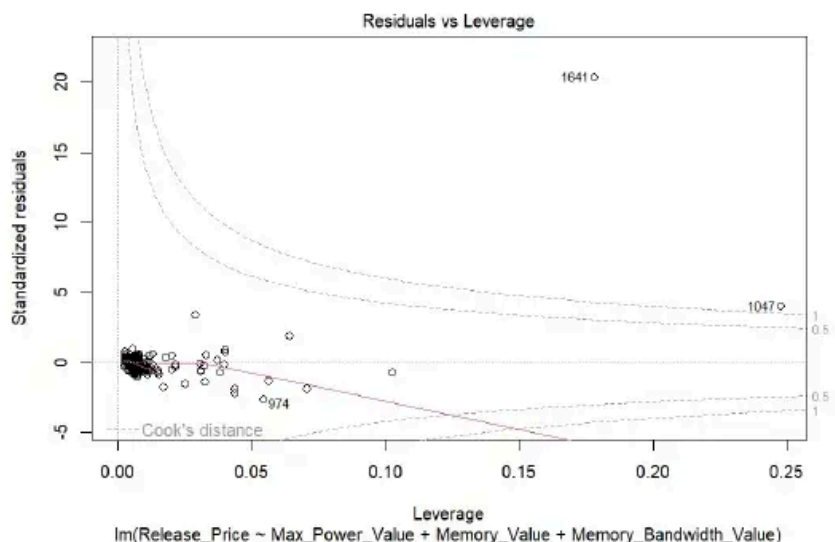
1. Tính tuyến tính của dữ liệu
2. Sai Số có kỳ vọng bằng 0
3. Phương sai của các số là hằng số
4. Sai số có phân phối chuẩn
5. Các sai số độc lập với nhau

Thực hiện phân tích thặng dư để kiểm tra các giả định của mô hình

```
plot(mo.hinh.3, 1)
plot(mo.hinh.3, 2)
plot(mo.hinh.3, 3)
plot(mo.hinh.3, 5)
```







Biểu đồ Residuals vs Fitted: Thể hiện các sai số tương ứng với các giá trị dự báo, kiểm tra giả định tuyến tính của dữ liệu, giả định sai số có kỳ vọng bằng 0, giả định phương sai của sai số là hằng số.

Dựa trên biểu đồ ta thấy, đường màu đỏ chưa phải là đường thẳng thực sự nằm ngang chứng tỏ tính tuyến tính của dữ liệu chưa cao.

Tuy nhiên, đường màu đỏ chưa nằm sát đường thẳng $y = 0$, nên giả định sai số có kỳ vọng bằng 0 chưa thoả mãn.

Các điểm sai số chưa phân tán ngẫu nhiên xung quanh đường màu đỏ nên giả định phương sai các sai số là hằng số cũng chưa thoả mãn.

Biểu đồ QQ Residuals: Thể hiện các sai số đã được chuẩn hoá, kiểm tra giả định sai số có phân phối chuẩn.

Dựa trên biểu đồ ta thấy, phần lớn các điểm sai số nằm trên đường thẳng kỳ vọng của phân phối chuẩn, do đó giả định sai số có phân phối chuẩn có thể xem là thoả mãn

Biểu đồ Scale-Location: Thể hiện căn bậc hai của các sai số đã được chuẩn hoá, kiểm tra giả định phương sai các sai số là hằng số.

Dựa trên biểu đồ ta thấy, các điểm sai số không phân tán đều xung quanh đường màu đỏ và đường màu đỏ có độ dốc. Do đó, giả định phương sai của các sai số là hằng số và giả định các sai số độc lập với nhau không thoả mãn.

Biểu đồ Residuals vs Leverage: Thể hiện các điểm gây ảnh hưởng cao trong bộ dữ liệu.

Dựa trên biểu đồ ta thấy, các điểm gây ảnh hưởng cao trong bộ dữ liệu bao gồm 947, 1641, 1047. Tuy nhiên, các điểm này cũng không vượt ra khỏi đường Cook's distance nên những điểm này chưa thực sự là những điểm ngoại lai có gây ảnh hưởng trong bộ dữ liệu. Vì vậy, ta không cần loại bỏ những điểm này khi phân tích

Giá trị p - value $< 2.2e - 16$ cho thấy mô hình hồi quy khá tốt. Tuy nhiên, mô hình trên không thoả một số giả định nên kết quả mà mô hình dự đoán có thể không chính xác.



5.5.5 Dự đoán giá

Từ phương trình hồi quy đã tìm được trong mô hình 3, ta có thể dự đoán giá thành sản phẩm GPU với các thông số như sau: Max_Power_Value = 225 Watts, Memory_Value = 8190 MB, Memory_Bandwidth_Value = 70 GB/sec

```
assump_data <- data.frame(Max_Power_Value = 225, Memory_Value = 8190,  
  Memory_Bandwidth_Value = 70)  
predict(mo.hinh.3, assump_data, interval = "confidence")
```

fit	lwr	upr
1499.739	1216.558	1782.92

Từ kết quả dự đoán, giá thành của GPU tương ứng với các thông số trên sẽ là \$1499.739 và khoảng ước lượng cho giá ra mắt thực sự là (\$1216.558; \$1782.92).

6 Tổng kết

Nhóm đã thành công trong việc nghiên cứu các lý thuyết về mô hình hồi quy tuyến tính đơn, hồi quy tuyến tính bội; tìm hiểu các lệnh của R để thực hiện việc tính toán, xây dựng mô hình, kiểm định giả thiết cho mô hình. Hơn nữa, từ mô hình đã xây dựng, nhóm cũng đã đưa ra được những dự đoán cho biến phụ thuộc.

Tuy nhiên, kết quả nhóm trình bày có thể chưa thật sự chính xác. Nguyên nhân có thể đến từ nhiều yếu tố:

- Tỷ lệ khuyết dữ liệu lớn, ảnh hưởng của việc loại bỏ các quan sát khuyết trong quá trình tiền xử lý số liệu.
- Sự hạn chế về mặt dữ liệu, các mẫu dữ liệu không thoả mãn các giả thiết cần của mô hình, dẫn đến kết quả chưa thật sự có ý nghĩa lớn về mặt thống kê.

Chính vì những hạn chế trên, nhìn chung kết quả mà nhóm tìm được có thể có sai khác so với kết quả chính xác.

7 Thảo luận và mở rộng

Bài tập lớn này đã giúp nhóm có cơ hội được áp dụng được những kiến thức lý thuyết đã học từ môn Xác suất thống kê ở học kỳ này vào ứng dụng thực tế; học hỏi và nghiên cứu các nội dung cơ bản trong lĩnh vực thống kê. Cùng với đó, nhóm cũng đã tìm tòi và biết cách áp dụng ngôn ngữ R cho việc xử lý thống kê, từ đó thấy được sức mạnh của công cụ này trong việc hỗ trợ tính toán.

Do giới hạn về mặt kiến thức, thời gian và khả năng làm việc nhóm, bài tập này không thể tránh khỏi những sai sót. Nhóm sẽ ghi nhận, khắc phục và lấy đó làm kinh nghiệm cho những bài tập sau này.



Tài liệu

- [1] Nguyễn Kiều Dung, Slides bài giảng Xác suất thống kê.
- [2] Nguyễn Đình Huy, Giáo trình Xác suất thống kê, NXB ĐHQG TP.HCM.
- [3] Peter Dalgaard, Introductory Statistics with R, Springer, 2008.
- [4] G. Jay Kerns, Introduction to Probability and Statistics Using R, 2nd ed., CRC, 2015.